[University Name]
[Faculty / Department]

MASTER'S THESIS

Study on Design and Implementation of a Lightweight Single-Class Object Detection Deep Neural Network

Submitted in partial fulfilment of the requirements for the degree of
Master of Science in Computer Science and Technology

Author: [Student Name]
Supervisor: [Supervisor Name]

2026

Abstract

Object detection is a core task in computer vision and underpins applications such as video surveillance, autonomous driving, industrial inspection and pedestrian analysis. Modern detectors based on deep convolutional neural networks achieve high accuracy, but they are usually designed for multi-class benchmarks such as MS COCO and carry a computational cost that is too high for embedded and edge devices. In many practical deployments, however, only a single object category is of interest — for example pedestrians, vehicles, faces or a specific industrial defect. A general multi-class detector wastes capacity on categories that never occur and on classification branches that are not required.

This thesis studies the design and implementation of a lightweight single-class object detection network (referred to as LSD-Net, Lightweight Single-class Detector) that targets real-time inference on resource-constrained hardware while preserving competitive accuracy. The work makes four contributions. First, a compact backbone is designed from depthwise separable convolutions, inverted residual blocks and a channel attention module, reducing parameters and floating-point operations (FLOPs) substantially compared with standard backbones. Second, a lightweight bidirectional feature pyramid neck with reduced channel width is proposed to fuse multi-scale features at low cost. Third, an anchor-free, single-class detection head is designed in which the classification branch is reduced to an objectness score, and the box regression branch is trained with a Complete-IoU (CIoU) loss. Fourth, a training and compression pipeline — including mosaic augmentation, label assignment based on centre sampling, knowledge distillation from a larger teacher model and post-training quantisation to INT8 — is developed to maximise accuracy per unit of computation.

The proposed method is evaluated on a single-class pedestrian detection task derived from the CrowdHuman and COCO-person datasets, and compared with lightweight baselines such as YOLOv5-n, YOLOv8-n, NanoDet and MobileNetV2-SSDLite. Ablation experiments are designed to measure the contribution of each component. The evaluation protocol reports average precision (AP@0.5 and AP@0.5:0.95), parameter count, FLOPs, model size and frames per second (FPS) on both a desktop GPU and an embedded device.

Keywords: object detection; lightweight neural network; single-class detection; depthwise separable convolution; anchor-free detection; knowledge distillation; model quantisation; edge computing.

Table of Contents

Abstract **2**

Chapter 1 Introduction **5**

1.1 Research Background 5

1.2 Research Significance 5

1.3 Research Status at Home and Abroad 6

1.4 Research Objectives and Content 17

1.5 Thesis Organisation 17

Chapter 2 Theoretical Foundations **18**

2.1 Convolutional Neural Networks 18

2.2 Training of Neural Networks 18

2.3 Normalisation and Activation Functions 19

2.4 Depthwise Separable Convolution 20

2.5 Inverted Residual Blocks and Attention 21

2.6 Object Detection Framework 22

2.7 Evaluation Metrics 24

2.8 Chapter Summary 25

Chapter 3 Design of the Lightweight Single-Class Detection Network **26**

3.1 Design Principles 26

3.2 Overall Architecture 26

3.3 Lightweight Backbone 27

3.4 Lightweight Feature Fusion Neck 31

3.5 Anchor-Free Single-Class Detection Head 32

3.6 Loss Function 33

3.7 Label Assignment 35

3.8 Complexity Analysis 36

3.9 Design Alternatives Considered 38

3.10 Summary of Architecture Hyper-Parameters 39

3.11 Chapter Summary 39

Chapter 4 Training, Distillation and Quantisation Strategy **40**

4.1 Dataset Construction 40

4.2 Data Augmentation 40
4.3 Training Settings 40
4.4 Knowledge Distillation 41
4.5 Post-Training Quantisation 43
4.6 Deployment Pipeline 46
4.7 Chapter Summary 47

Chapter 5 Experiments and Analysis **48**

5.1 Experimental Environment 48
5.2 Evaluation Protocol 48
5.3 Comparison with Lightweight Detectors 48
5.4 Ablation Study 49
5.5 Quantisation Results 53
5.6 Qualitative Analysis 55
5.7 Discussion 56
5.8 Chapter Summary 56

Chapter 6 Conclusion and Future Work **57**

6.1 Conclusions 57
6.2 Limitations 57
6.3 Future Work 57

References **59**

Acknowledgements **64**

Chapter 1
Introduction

1.1 Research Background

Object detection is the problem of localising every instance of objects of interest in an image and assigning each a category label. It is one of the most important problems in computer vision because it bridges low-level image understanding and high-level semantic reasoning. The rapid progress of deep learning since 2012 has transformed detection from a field dominated by hand-crafted features such as Histograms of Oriented Gradients (HOG) and Deformable Part Models (DPM) into one dominated by convolutional neural networks (CNNs) and, more recently, transformers.

Two families of deep detectors have emerged. Two-stage detectors such as R-CNN, Fast R-CNN and Faster R-CNN first generate region proposals and then classify and refine each proposal. One-stage detectors such as YOLO, SSD, RetinaNet and FCOS predict boxes and classes densely in a single forward pass. Two-stage detectors historically offered higher accuracy, while one-stage detectors offered higher speed; in recent years, careful engineering of one-stage detectors (e.g., the YOLO series) has largely closed the accuracy gap.

Despite these advances, state-of-the-art detectors remain computationally expensive. A typical high-accuracy detector requires tens of billions of FLOPs per image and tens of millions of parameters. Such models run comfortably on server GPUs but are unsuitable for embedded platforms such as the NVIDIA Jetson series, Raspberry Pi, mobile phones or microcontroller-class devices, where memory, power and thermal budgets are strictly limited.

At the same time, a large proportion of real-world detection applications are single-class problems. A people-counting camera only needs to detect people; a helmet-compliance system detects helmets; a fruit-picking robot detects one type of fruit; a production-line inspector detects one type of defect. For these applications, the multi-class machinery of general detectors is redundant. This observation motivates the central question of this thesis: *how can a detection network be designed specifically for the single-class setting so that it is as small and fast as possible while retaining high accuracy?*

1.2 Research Significance

The significance of this research is both theoretical and practical.

- **Theoretical significance.** Studying single-class detection isolates the localisation problem from the multi-class classification problem. This allows a clearer analysis of which architectural components contribute to localisation accuracy, and of how much network capacity is really required for one category.
- **Practical significance.** A lightweight single-class detector can be deployed on low-cost edge hardware, reducing system cost, energy consumption, network bandwidth (because video need not be streamed to the cloud) and privacy risk (because raw images stay on the device).
- **Engineering significance.** The thesis provides a complete, reproducible pipeline from architecture design to training, compression and deployment, which can be reused for other single-class tasks.

1.3 Research Status at Home and Abroad

1.3.1 Development of general object detectors

Girshick et al. introduced R-CNN, which combined selective-search region proposals with CNN features and SVM classifiers. Fast R-CNN shared computation through RoI pooling, and Faster R-CNN replaced selective search with a learnable Region Proposal Network (RPN), achieving near real-time performance. Redmon et al. proposed YOLO, which framed detection as a single regression problem over a grid; subsequent versions (YOLOv2 to YOLOv8 and beyond) added anchor boxes, multi-scale prediction, CSP backbones, PANet necks and anchor-free decoupled heads. Liu et al. proposed SSD, which predicts from multiple feature maps of different resolutions. Lin et al. introduced Focal Loss and RetinaNet to address the extreme foreground–background imbalance of dense detectors. Tian et al. proposed FCOS, an anchor-free detector that predicts distances from each location to the four sides of the box. Carion et al. proposed DETR, which uses a transformer and bipartite matching to remove hand-designed components such as non-maximum suppression (NMS).

1.3.2 Historical context: from hand-crafted features to deep detectors

Before the deep learning era, the dominant pipeline combined a sliding-window search with hand-crafted descriptors and a shallow classifier. Dalal and Triggs [67] showed that Histograms of Oriented Gradients combined with a linear support vector machine gave a large improvement in pedestrian detection, and this descriptor became the reference feature for a decade. Felzenszwalb et al. [68] extended the idea with Deformable Part Models, which represent an object as a root filter plus a set of deformable part filters and were the winning approach on PASCAL VOC [70] for several years. The benchmark study of Dollár et al. [28] documented the progress of such methods on the Caltech pedestrian dataset and introduced the log-average miss rate that is still used today. These

methods are important for this thesis for two reasons: they were already single-class detectors in most applications, and their computational profile — a cheap feature extractor plus a cheap per-window classifier — is the historical precursor of lightweight dense detectors.

1.3.3 Two-stage detectors: the R-CNN family

R-CNN [1] established the region-based paradigm. Approximately two thousand category-independent proposals are generated by selective search, each proposal is warped to a fixed size and passed through a CNN pre-trained on ImageNet, and the resulting features are classified by per-class linear SVMs, followed by class-specific bounding-box regression. Although R-CNN improved the mean average precision on PASCAL VOC by a large margin, it was slow because the CNN was evaluated independently for every proposal, and its multi-stage training was cumbersome.

Fast R-CNN [2] computes the convolutional feature map once per image and extracts a fixed-size feature for every proposal with RoI pooling. Classification and regression are learned jointly with a multi-task loss that combines softmax cross-entropy and a smooth-L1 regression loss, making training single-stage and end-to-end with respect to the detection network. The remaining bottleneck was the external proposal algorithm. Faster R-CNN [3] removed it with the Region Proposal Network, a small fully convolutional network that slides over the shared feature map and predicts objectness and box offsets for a set of reference boxes — the anchors — at every position. Anchors of three scales and three aspect ratios introduced the anchor mechanism that later dominated one-stage detection.

Subsequent work refined the two-stage design along several axes. Feature Pyramid Networks [7] added a top-down pathway with lateral connections so that proposals of different sizes are pooled from pyramid levels of appropriate resolution, which greatly improved small-object detection. Mask R-CNN [36] replaced RoI pooling with RoIAlign, which avoids coordinate quantisation by bilinear sampling, and added an instance mask branch. Cascade R-CNN [37] observed that a detector trained at a single IoU threshold is optimal only for proposals of similar quality, and therefore trained a sequence of heads with increasing IoU thresholds (0.5, 0.6, 0.7), each stage refining the boxes produced by the previous one; this produced substantial gains at high IoU thresholds.

From the perspective of lightweight single-class detection, two-stage detectors have three drawbacks. Their per-region head is executed hundreds of times per image, so latency grows with the number of proposals; the RoI operations are irregular memory accesses that are poorly supported by many embedded inference engines; and the

second-stage classifier, whose main role in a multi-class setting is fine-grained category discrimination, contributes relatively little when only one category exists. Nevertheless, the ideas of multi-scale pyramids (FPN) and quality-progressive refinement (Cascade R-CNN) have been carried over to one-stage designs and inform the neck and quality-score design of this thesis.

1.3.4 One-stage detectors: the YOLO series

YOLO (v1) [4] divided the image into an S x S grid (S = 7) and let each cell predict B boxes with confidence scores together with one set of class probabilities. The entire detector was a single network trained with a sum-of-squared-errors loss and ran at 45 FPS on a desktop GPU, but its coarse grid limited recall for small and adjacent objects, since each cell could predict only one class and few boxes.

YOLOv2 (YOLO9000) [29] introduced batch normalisation in all convolutional layers, a high-resolution classifier pre-training stage, anchor boxes whose shapes were obtained by k-means clustering of training boxes with a $1 - \text{IoU}$ distance, direct location prediction constrained by a sigmoid to the cell, a pass-through layer for fine-grained features and multi-scale training. YOLOv3 [30] adopted the Darknet-53 backbone with residual connections, predicted at three scales in an FPN-like manner, and replaced softmax by independent logistic classifiers, which allowed multi-label outputs.

YOLOv4 [31] systematically evaluated a large collection of training techniques ("bag of freebies") and architectural modules ("bag of specials"). It combined a CSPDarknet-53 backbone, derived from Cross Stage Partial networks [73] that split the feature map into two parts and merge them after a dense block to reduce duplicated gradient information, with an SPP module, a PANet [47] neck that adds a bottom-up path to FPN, mosaic augmentation and the CIoU loss [19]. YOLOv5, released as an open-source engineering project rather than a paper, popularised a modular PyTorch implementation, automatic anchor fitting, the SPPF module and a family of depth and width multipliers (n, s, m, l, x), and its nano variant became a common lightweight baseline.

YOLOv6 [32] targeted industrial deployment with an EfficientRep backbone built from RepVGG-style re-parameterisable blocks [50], a decoupled head, task-aligned label assignment [43] and quantisation-friendly design. YOLOv7 [33] introduced the extended efficient layer aggregation network (E-ELAN), planned re-parameterised convolution and a coarse-to-fine lead-guided label assignment with auxiliary heads. YOLOv8, again released as software, moved to an anchor-free decoupled head, used task-aligned assignment together with distribution focal loss [20] and the CIoU loss, and replaced the C3 block with the lighter C2f block. YOLOv9 [34] proposed programmable gradient information and the GELAN architecture to mitigate information loss in deep networks.

YOLOv10 [35] introduced consistent dual assignments — a one-to-many branch for rich supervision during training and a one-to-one branch used at inference — which removes the need for NMS while keeping training efficient, together with a holistic efficiency-driven design of the head and down-sampling layers.

Three trends in this lineage are directly relevant to LSD-Net. First, anchors were introduced and then removed again, because anchor-free heads reduce hyper-parameters and output size without loss of accuracy. Second, label assignment evolved from fixed IoU thresholds to dynamic, prediction-aware strategies. Third, the classification and localisation branches were decoupled, but their scores were re-coupled through quality-aware targets. All three trends are adopted and specialised to the single-class case in Chapter 3.

1.3.5 One-stage detectors: SSD and RetinaNet

SSD [5] predicted category scores and box offsets for default boxes attached to several feature maps of decreasing resolution taken from a truncated VGG-16 backbone and extra layers. Matching used a Jaccard overlap threshold of 0.5, and the extreme imbalance between positive and negative default boxes was handled by hard negative mining, keeping a negative-to-positive ratio of at most 3:1. SSD was the first one-stage detector to rival Faster R-CNN in accuracy at much higher speed, and its MobileNet-based variant SSDLite, which replaces standard convolutions in the prediction layers by depthwise separable ones [11], remains a classic mobile baseline.

RetinaNet [6] attributed the accuracy gap between one- and two-stage detectors to class imbalance during training and proposed the focal loss, which down-weights the loss of well-classified examples so that the training signal is dominated by hard examples. Combined with an FPN backbone and nine anchors per location, RetinaNet surpassed contemporary two-stage detectors. The focal loss has since become the standard classification loss for dense detectors and is the parent of the Quality Focal Loss [20] and the Varifocal Loss [44] used in modern heads.

1.3.6 Anchor-free detectors: CornerNet, FCOS, CenterNet and YOLOX

Anchor-free detectors avoid the design of anchor scales and aspect ratios, which must otherwise be re-tuned for every dataset. CornerNet [39] detected an object as a pair of keypoints, the top-left and bottom-right corners, predicted as heatmaps and grouped by learned embeddings, with corner pooling to localise corners that lie outside the object. CenterNet ("Objects as Points") [38] simplified this further: an object is represented by its centre point on a single stride-4 heatmap, and the box size and a sub-pixel offset are regressed at the peak. Because duplicate peaks are suppressed by a 3 x 3 max-pooling operation, CenterNet can operate without box-level NMS.

FCOS [8] treats every location inside a ground-truth box as a potential positive and regresses the four distances to the box sides. Ambiguity between overlapping objects is resolved by assigning objects of different sizes to different FPN levels according to the maximum regression distance, and low-quality predictions far from object centres are suppressed by a "centre-ness" branch whose output multiplies the classification score. ATSS [42] then showed that much of the difference between anchor-based and anchor-free detectors is due to the definition of positive and negative samples, and proposed to select, for each object, the k closest candidates per level and set the IoU threshold adaptively as the mean plus standard deviation of their IoUs.

YOLOX [21] brought these ideas to the YOLO family: an anchor-free decoupled head, strong augmentation (mosaic and MixUp [71]) and SimOTA, a simplified optimal-transport label assignment that selects for each ground truth a dynamic number of lowest-cost predictions. YOLOX demonstrated that anchor-free heads with dynamic assignment are well suited to small models; its nano and tiny versions were competitive with much larger anchor-based detectors. TOOD [43] further proposed task-aligned learning, in which the assignment metric is the product of the classification score raised to a power and the IoU raised to another power, so that positives are those locations where both tasks are simultaneously good.

1.3.7 Transformer-based detectors: DETR and its variants

DETR [9] formulated detection as a direct set prediction problem. A CNN backbone and a transformer encoder produce image features; a transformer decoder transforms a fixed set of learned object queries into predictions, and a Hungarian bipartite matching between predictions and ground truths defines a one-to-one training target. Since each object is matched to exactly one prediction, duplicates are penalised during training and NMS is unnecessary. The original DETR, however, converged slowly (hundreds of epochs) and performed poorly on small objects because the attention over all feature-map positions is computationally prohibitive at high resolution.

Deformable DETR [40] replaced dense attention with multi-scale deformable attention, in which each query attends to only a small set of sampling points around a reference point on each pyramid level. This reduced complexity from quadratic to linear in the number of positions, improved small-object accuracy and shortened training by roughly an order of magnitude. RT-DETR [41] made DETR-style detection real-time by designing an efficient hybrid encoder that decouples intra-scale interaction (attention on the coarsest level only) from cross-scale fusion (convolutional), and by selecting initial queries according to an IoU-aware score. RT-DETR showed that an end-to-end transformer detector can outperform contemporary YOLO models at similar latency on

server GPUs.

For edge deployment, transformer detectors remain less attractive than convolutional ones. Attention, layer normalisation and the gather operations required by deformable sampling are less mature in embedded inference engines and in INT8 quantisation toolchains, and their memory traffic is high relative to their FLOP count. The key conceptual contribution of DETR — one-to-one matching that removes NMS — has nevertheless been transferred back to convolutional detectors, as in YOLOv10 [35], and is identified as future work for LSD-Net.

1.3.8 Lightweight network design

Lightweight backbone research seeks to reduce computation without large accuracy loss. MobileNetV1 introduced depthwise separable convolutions; MobileNetV2 added inverted residuals with linear bottlenecks; MobileNetV3 used neural architecture search together with squeeze-and-excitation (SE) attention and the h-swish activation. ShuffleNet and ShuffleNetV2 used group convolutions and channel shuffling and proposed practical guidelines for efficient design based on memory access cost. GhostNet generated "ghost" feature maps from cheap linear operations. EfficientNet proposed compound scaling of depth, width and resolution. These backbones have been combined with detection heads to produce lightweight detectors such as MobileNet-SSDLite, NanoDet, PP-PicoDet, YOLOv5-n and YOLOv8-n.

1.3.9 Lightweight backbones in detail

MobileNetV1 [10] built an entire network from depthwise separable convolutions and introduced two global hyper-parameters: a width multiplier that thins every layer uniformly and a resolution multiplier that reduces the input size. MobileNetV2 [11] observed that applying ReLU in a low-dimensional manifold destroys information, and therefore proposed the inverted residual with linear bottleneck: the shortcut connects thin bottleneck layers, while the non-linear depthwise convolution operates in an expanded space (expansion factor 6 by default). MobileNetV3 [12] combined platform-aware neural architecture search with the NetAdapt algorithm to find a block configuration optimised for mobile CPU latency, inserted lightweight SE attention [17] into selected blocks, and introduced the hard-swish activation, $\text{h-swish}(x) = x \cdot \text{ReLU6}(x + 3) / 6$, a piecewise-linear approximation of the Swish function [72] that is cheap to compute and quantise.

ShuffleNet [48] reduced the cost of the dense 1 x 1 convolutions — which dominate the cost of separable networks — by using pointwise group convolutions, and used a channel shuffle operation to restore information exchange between groups. ShuffleNetV2 [13] argued that FLOPs are an indirect metric and derived four practical guidelines from

latency measurements: equal input and output channel widths minimise memory access cost (MAC); excessive group convolution increases MAC; network fragmentation reduces parallelism; and element-wise operations are not negligible. Its basic unit splits the channels into two halves, processes only one of them and concatenates and shuffles the result.

GhostNet [14] observed that many feature maps in a trained CNN are similar to each other, and generated part of the output channels ("ghost" features) from the remainder by cheap depthwise operations. EfficientNet [15] showed that depth, width and input resolution should be scaled together with a compound coefficient, and obtained a family of backbones (B0 to B7) from a searched B0 baseline built from MBConv blocks with SE attention. MobileViT [49] combined MobileNetV2 blocks with a light transformer block that unfolds the feature map into non-overlapping patches, applies self-attention across patches and folds the result back, thereby giving a mobile network a global receptive field.

RepVGG [50] introduced structural re-parameterisation. During training each block contains three parallel branches — a 3 x 3 convolution, a 1 x 1 convolution and an identity — each followed by BN. Because convolution and BN are linear at inference time, the three branches can be merged algebraically into a single 3 x 3 convolution: the 1 x 1 kernel is zero-padded to 3 x 3, the identity is expressed as a 3 x 3 kernel with a single one at the centre of the corresponding channel, and each BN is folded into its kernel and bias. The deployed network is therefore a plain stack of 3 x 3 convolutions and ReLUs, which is extremely fast on GPUs, while training benefits from the multi-branch topology. Re-parameterisation is used by YOLOv6, YOLOv7 and PP-PicoDet; its implications for depthwise blocks are discussed in Chapter 3.

1.3.10 Lightweight detectors

Several detectors have been designed specifically for mobile and edge devices. EfficientDet [16] combined EfficientNet backbones with the bidirectional feature pyramid (BiFPN), which removes nodes with a single input, adds skip edges between nodes of the same level and uses fast normalised weighted fusion; its smallest model, EfficientDet-D0, uses a 512 x 512 input and a 64-channel BiFPN with three repeated layers. While D0 is very efficient in FLOPs, its depthwise-heavy BiFPN is less efficient in latency on GPUs than its FLOP count suggests.

NanoDet, an open-source project, combined a ShuffleNetV2 backbone with a PAN neck that uses only interpolation and addition, a shared depthwise head and the Generalized Focal Loss [20]; NanoDet-Plus added a ghost-module PAN and a dynamic assignment guided by an auxiliary training-only head. PP-PicoDet [51] used an ESNet backbone

obtained by neural architecture search from ShuffleNetV2-style blocks with SE attention, a CSP-PAN neck, a SimOTA-based assignment with Varifocal loss and GIoU loss, and was optimised with the deployment toolchain of its framework. The nano variants of YOLOv5, YOLOv6, YOLOv8 and later YOLO releases obtain a lightweight model by uniformly reducing depth and width multipliers of the full model. BlazeFace [26] is a notable single-class example: designed for front-facing camera faces, it uses BlazeBlocks with 5 x 5 depthwise kernels, an anchor scheme specialised to the near-constant aspect ratio of faces and a blending strategy instead of NMS to reduce temporal jitter.

A common property of these models is that they are designed for multi-class benchmarks (typically COCO with 80 classes) and are adapted to single-class tasks only by changing the number of output channels. The architecture itself — channel widths, head depth and assignment — is not re-balanced for the reduced difficulty of the classification sub-task.

1.3.11 Model compression

Complementary to architecture design, model compression techniques reduce the cost of an existing network. Pruning removes redundant weights or channels; quantisation represents weights and activations with low-bit integers; knowledge distillation, proposed by Hinton et al., trains a small student network to mimic a large teacher. Distillation for detection is more complex than for classification because of the dense, imbalanced outputs, and has been studied through feature imitation, response distillation and localisation distillation.

1.3.12 Model compression for detection in detail

Pruning. Han et al. [52] showed that a large fraction of the weights of a trained network can be removed by magnitude-based unstructured pruning and combined with quantisation and Huffman coding ("deep compression"). Unstructured sparsity, however, requires dedicated sparse kernels to accelerate inference. Structured pruning removes whole filters or channels and directly yields a smaller dense network. Li et al. [54] ranked filters by their L1 norm, while Network Slimming [53] imposed an L1 penalty on the BN scaling factors during training and removed channels whose factors approach zero. Structured pruning is widely applied to YOLO models, but care is required at residual connections and concatenations, where coupled channels must be pruned consistently.

Quantisation. Jacob et al. [23] proposed an affine integer quantisation scheme with integer-only arithmetic at inference time, and quantisation-aware training with simulated quantisation in the forward pass. Krishnamoorthi [69] systematically compared per-tensor and per-channel quantisation and post-training versus quantisation-aware

approaches, and found that per-channel weight quantisation is essential for depthwise convolutions, whose per-channel weight ranges vary greatly. This observation is directly relevant to LSD-Net, which relies heavily on depthwise convolutions.

Knowledge distillation for detection. Hinton et al. [22] distilled softened class probabilities from a teacher to a student, while FitNets [58] showed that intermediate feature maps can also serve as hints. Chen et al. [55] were among the first to apply distillation to multi-class detection, combining hint learning on features with weighted cross-entropy on classifier outputs and a teacher-bounded regression loss. Wang et al. [56] observed that background regions dominate feature imitation and proposed fine-grained feature imitation restricted to the anchor locations near objects. Localisation distillation [57] transferred the teacher's localisation knowledge by distilling the discretised box-edge distributions of a GFL-style head, showing that localisation knowledge is as important as classification knowledge for detection. In the single-class setting the classification knowledge is reduced to a single score, so feature and localisation knowledge become relatively more important.

1.3.13 Single-class detection

Single-class detection has been studied mainly in application-specific settings: pedestrian detection (Caltech, CityPersons, CrowdHuman), face detection (WIDER FACE, with detectors such as RetinaFace and BlazeFace), and vehicle detection. Face detectors such as BlazeFace demonstrated that a carefully designed single-class detector can run at hundreds of FPS on mobile GPUs. However, there is little systematic study of how general detector design choices should be adapted when the number of classes is reduced to one.

1.3.14 Single-class detection domains

Pedestrian detection. Pedestrian detection has a long history with its own benchmarks. The Caltech benchmark [28] focuses on small pedestrians in urban driving video; CityPersons [63] provides higher-quality annotations on Cityscapes images with a larger diversity of cities and occlusion levels; CrowdHuman [24] contains on average more than twenty persons per image with heavy mutual occlusion, and provides full-body, visible-body and head boxes. Key difficulties are the large range of scales, partial occlusion and the crowd problem: adjacent pedestrians produce highly overlapping ground-truth boxes, so a standard NMS threshold either suppresses true positives or retains duplicates. The standard metric is the log-average miss rate, often reported for subsets defined by height and visibility.

Face detection. WIDER FACE [64] contains faces with large variations in scale, pose and occlusion and defines easy, medium and hard subsets. RetinaFace [65] is a

single-stage face detector that adds five-point landmark regression and a self-supervised mesh decoder as auxiliary tasks, and offers a MobileNet-0.25 backbone for real-time CPU inference. BlazeFace [26] reached sub-millisecond inference on mobile GPUs by exploiting the restricted geometry of selfie faces. Face detection shows that strong domain priors — aspect ratio, typical scale, auxiliary landmarks — can be exploited by a single-class detector.

Vehicle detection. Vehicle detection is studied on driving datasets such as KITTI [66], where cars, vans and trucks are often merged into a single "vehicle" class for traffic monitoring applications. Vehicles exhibit large aspect-ratio changes with viewpoint, strong truncation at image borders and occlusion in congestion. Compared with pedestrians, they are rigid but more varied in aspect ratio, which affects the choice of assignment strategy and of the regression loss.

1.3.15 Critical comparison of representative detectors

Table 1.1 summarises the representative detector families qualitatively from the perspective of single-class deployment on edge devices. The comparison is deliberately qualitative: published accuracies are measured on different benchmarks with different training schedules, and the quantitative comparison under a common protocol is the subject of Chapter 5.

Family	**Representative**	**Anchors**	**NMS**	**Edge friendliness**	**Main limitation for single-class edge use**
Two-stage	Faster R-CNN, Cascade R-CNN	Yes	Yes	Low	Per-RoI head, irregular RoI operations
One-stage anchor-based	SSD, RetinaNet, YOLOv3–v5	Yes	Yes	Medium	Anchor tuning, many outputs per location
Anchor-free dense	FCOS, YOLOX, YOLOv8	No	Yes	High	Multi-class head redundant for one class
Keypoint-based	CornerNet, CenterNet	No	Optional	Medium	High-resolution heatmap is costly

Family	Representative	Anchors	NMS	Edge friendliness	Main limitation for single-class edge use
Transformer	DETR, Deformable DETR, RT-DETR	No	No	Low–medium	Attention and gather ops poorly supported in INT8
Lightweight dense	NanoDet, PP-PicoDet, YOLO-n	Mostly no	Yes	High	Scaled-down multi-class design, not re-balanced

Table 1.1 Qualitative comparison of detector families for single-class edge deployment.

1.3.16 Gap analysis

The review above reveals the following gaps that motivate this thesis.

- **Capacity allocation.** Lightweight detectors are produced by uniformly shrinking multi-class models. When the number of classes is reduced from 80 to 1, the head's classification sub-task becomes much easier, whereas localisation remains as difficult as before. No surveyed lightweight detector re-allocates capacity from classification to localisation and multi-scale context accordingly.
- **Score semantics.** In single-class detection the classification score serves almost exclusively as a ranking signal for NMS and AP computation. Quality-aware scores [20][44] are therefore particularly appropriate, but they have been studied mainly as add-ons to multi-class heads.
- **Neck cost.** Concatenation-based PAN necks dominate the cost of many nano detectors at low resolution. Weighted additive fusion [16] reduces FLOPs, but its latency benefit on edge hardware and its effect on single-class accuracy have not been quantified.
- **Compression pipeline.** Distillation, quantisation and architecture design are usually studied in isolation. Depthwise-heavy architectures are known to be sensitive to per-tensor quantisation [69], yet few works evaluate the complete design–distillation–quantisation–deployment chain on embedded hardware with end-to-end latency including pre- and post-processing.
- **Crowded scenes.** Single-class domains such as pedestrians are characterised by dense crowds, where NMS and fixed assignment strategies fail; the interaction between dynamic assignment, quality scores and NMS variants in lightweight models deserves systematic study.

1.3.17 Summary of existing problems

From this review, three gaps are identified: (1) general lightweight detectors retain multi-class components that are redundant for single-class tasks; (2) the trade-off between neck complexity and accuracy in the single-class setting has not been studied systematically; (3) few works combine architecture design, distillation and quantisation into one end-to-end pipeline evaluated on embedded hardware.

1.4 Research Objectives and Content

The objective of this thesis is to design, implement and evaluate a lightweight single-class object detection network that achieves real-time inference on embedded hardware with accuracy close to that of larger general-purpose detectors. The research content is:

- Design of a lightweight backbone based on depthwise separable convolution, inverted residual blocks and efficient channel attention.
- Design of a lightweight multi-scale feature fusion neck.
- Design of an anchor-free single-class detection head and an appropriate loss function.
- Development of a training strategy including data augmentation, label assignment and knowledge distillation.
- Model compression by INT8 quantisation and deployment on an embedded device.
- Comprehensive comparative and ablation experiments.

1.5 Thesis Organisation

Chapter 1 introduces the background, significance, related work and objectives. Chapter 2 presents the theoretical foundations of CNNs, object detection and lightweight design. Chapter 3 describes the design of the proposed LSD-Net architecture. Chapter 4 describes the training, distillation and quantisation strategy. Chapter 5 presents the experimental design, results and analysis. Chapter 6 summarises the thesis and outlines future work.

Chapter 2
Theoretical Foundations

2.1 Convolutional Neural Networks

A convolutional neural network is composed of convolutional layers, normalisation layers, non-linear activations and pooling or strided operations. For an input feature map with C_{in} channels and spatial size H x W, a standard convolution with kernel size k and C_{out} output channels has the following computational cost:

$$FLOPs_{std} = H \cdot W \cdot C_{in} \cdot C_{out} \cdot k^2 \quad (2.1)$$

and $k^2 C_{in} C_{out}$ parameters. Batch normalisation (BN) normalises activations by mini-batch statistics, stabilising training and enabling higher learning rates. Common activations include ReLU, Leaky ReLU, SiLU (Swish) and h-swish. Residual connections, introduced in ResNet, add the input of a block to its output and enable the training of very deep networks.

2.2 Training of Neural Networks

2.2.1 Back-propagation

A network f(x; θ) with parameters θ is trained by minimising an empirical risk $L(\theta) = (1/N) \Sigma_n \ell(f(x_n; \theta), y_n)$ over the training set. The gradient of L with respect to every parameter is obtained efficiently by back-propagation [62], which applies the chain rule in reverse topological order of the computation graph. For a layer computing z = W a + b followed by an activation a' = g(z), the local gradients are:

$$\partial L/\partial W = \delta \cdot a^T,\ \partial L/\partial b = \delta,\ \partial L/\partial a = W^T \delta,\ \text{with } \delta = \partial L/\partial z = (\partial L/\partial a') \odot g'(z) \quad (2.2)$$

For a convolutional layer the same relations hold with the matrix product replaced by convolution: the weight gradient is the correlation of the input feature map with the output gradient, and the input gradient is the transposed convolution of the output gradient with the kernel. The cost of the backward pass is approximately twice that of the forward pass, which is why training a lightweight network is still considerably more expensive than inference.

2.2.2 Optimisers

Stochastic gradient descent (SGD) with momentum updates the parameters with a velocity term v that accumulates past gradients:

$$v_t = \mu\, v_{t-1} + g_t + \lambda\, \theta_{t-1},\ \theta_t = \theta_{t-1} - \eta_t\, v_t \quad (2.3)$$

where g_t is the mini-batch gradient, μ the momentum coefficient, λ the L2 weight-decay coefficient and η_t the learning rate. Nesterov momentum evaluates the gradient at the look-ahead position and often gives slightly faster convergence. Adam [60] maintains exponential moving averages of the first moment m_t and the second moment s_t of the gradient and uses bias-corrected estimates to compute a per-parameter step:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\, g_t,\; s_t = \beta_2 s_{t-1} + (1 - \beta_2)\, g_t^{\,2} \tag{2.4}$$

$$\theta_t = \theta_{t-1} - \eta\, \hat{m}_t / (\sqrt{\hat{s}_t} + \varepsilon),\; \hat{m}_t = m_t / (1 - \beta_1^{\,t}),\; \hat{s}_t = s_t / (1 - \beta_2^{\,t}) \tag{2.5}$$

Loshchilov and Hutter [61] showed that L2 regularisation and weight decay are not equivalent for adaptive methods, and proposed AdamW, in which the decay term $\lambda\theta$ is applied directly to the parameters rather than added to the gradient. In detection, SGD with momentum remains the most common choice for CNN detectors trained from ImageNet-pretrained backbones, whereas AdamW is standard for transformer detectors. Both are compared in the hyper-parameter study of Chapter 5.

2.2.3 Learning-rate schedules and weight averaging

Large initial learning rates destabilise the first iterations, particularly when BN statistics are not yet reliable, so a linear warm-up from a small value is used. The cosine schedule then decays the rate smoothly:

$$\eta_t = \eta_{min} + \tfrac{1}{2}\,(\eta_{max} - \eta_{min})\,(1 + \cos(\pi\, t / T)) \tag{2.6}$$

where T is the total number of iterations. An exponential moving average (EMA) of the weights, $\theta^{\mathrm{EMA}} \leftarrow d\,\theta^{\mathrm{EMA}} + (1 - d)\,\theta$, is evaluated instead of the raw weights; it acts as a temporal ensemble and typically yields a more accurate and more stable model at no inference cost.

2.3 Normalisation and Activation Functions

2.3.1 Batch normalisation

Batch normalisation [59] normalises each channel of a mini-batch B with its mean μ_B and variance $\sigma_B^{\,2}$ and then applies a learnable affine transformation:

$$\hat{x} = (x - \mu_B) / \sqrt{(\sigma_B^{\,2} + \varepsilon)},\; y = \gamma\, \hat{x} + \beta \tag{2.7}$$

During training, running estimates of the mean and variance are accumulated and used at inference time. BN reduces sensitivity to initialisation, permits larger learning rates and has a mild regularising effect. Its dependence on batch statistics makes it unreliable for very small per-GPU batches, which is one reason why detectors are commonly trained with synchronised BN across devices or with sufficiently large batches. At inference time BN is a fixed per-channel affine transformation and can be folded into the

preceding convolution with kernel W and bias b:

$$W' = (\gamma / \sqrt{(\sigma^2 + \varepsilon)}) \cdot W,\ b' = (\gamma / \sqrt{(\sigma^2 + \varepsilon)}) \cdot (b - \mu) + \beta \quad (2.8)$$

Folding removes all BN computation and memory traffic at deployment and is a prerequisite for integer-only inference. The scaling factors γ are also used as channel importance indicators in network slimming [53].

2.3.2 Activation functions

ReLU, max(0, x), is cheap and produces sparse activations but has zero gradient for negative inputs. ReLU6 clips the output at 6, which bounds the activation range and therefore improves robustness to low-precision arithmetic; it was used in MobileNetV1 and V2. Leaky ReLU passes a small slope for negative inputs. Swish, $x \cdot \sigma(\beta x)$ [72], also known as SiLU when $\beta = 1$, is smooth and non-monotonic and improves accuracy in deep networks, but the sigmoid is relatively expensive on embedded hardware. Hard-swish [12] approximates Swish with a piecewise-linear function:

$$h\text{-}swish(x) = x \cdot ReLU6(x + 3) / 6 \quad (2.9)$$

which is cheap to compute and easy to fuse into integer pipelines. Because the activation is executed at every position of every expanded feature map, the choice of activation has a measurable effect on edge latency even though it contributes few FLOPs; this effect is evaluated in the ablation study.

2.4 Depthwise Separable Convolution

A depthwise separable convolution factorises a standard convolution into a depthwise convolution, which applies one k x k filter per input channel, followed by a pointwise 1 x 1 convolution that mixes channels. Its cost is:

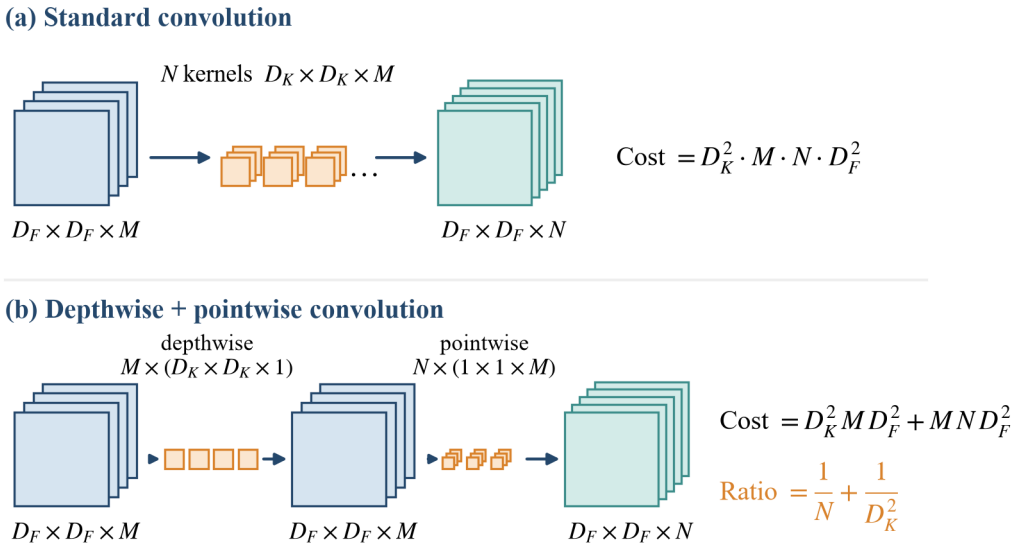


Figure 2.1 Standard convolution versus depthwise + pointwise (depthwise separable) convolution, with their computational costs and cost ratio.

$$FLOPs_{dw} = H \cdot W \cdot C_{in} \cdot k^2 + H \cdot W \cdot C_{in} \cdot C_{out} \tag{2.10}$$

The ratio of the cost of the separable convolution to the standard convolution is therefore:

$$FLOPs_{dw} / FLOPs_{std} = 1 / C_{out} + 1 / k^2 \tag{2.11}$$

For k = 3 and typical channel widths, this yields an 8–9x reduction in computation, which is the foundation of most lightweight architectures.

2.5 Inverted Residual Blocks and Attention

The inverted residual block of MobileNetV2 first expands a narrow input with a 1 x 1 convolution by an expansion factor t, applies a 3 x 3 depthwise convolution in the high-dimensional space, and then projects back to a narrow output with a linear 1 x 1 convolution. A residual connection joins the narrow input and output when their shapes match. Because the non-linearity is applied in the expanded space and the projection is linear, information loss caused by ReLU in low-dimensional spaces is reduced.

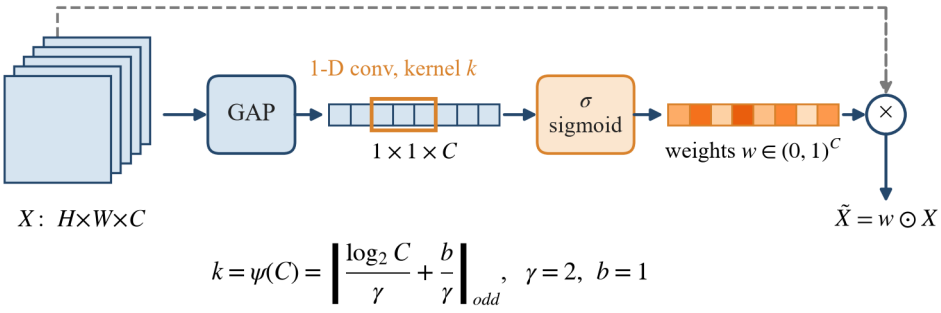


Figure 2.2 ECA module: global average pooling, a 1-D convolution with adaptive kernel size k, sigmoid gating and channel-wise rescaling of the input feature map.

Channel attention re-weights feature channels according to their global importance. The squeeze-and-excitation (SE) module computes global average pooled descriptors and passes them through two fully connected layers. The Efficient Channel Attention (ECA) module replaces the fully connected layers by a 1-D convolution of size k across channels:

$$w = \sigma(\, Conv1D_k(\, GAP(X)\,)\,),\; Y = w \odot X \tag{2.12}$$

where GAP is global average pooling, σ is the sigmoid function and ⊙ is channel-wise multiplication. ECA adds only k parameters per module, which makes it well suited to lightweight networks.

2.6 Object Detection Framework

2.6.1 Components of a one-stage detector

A modern one-stage detector consists of three parts: a **backbone** that extracts hierarchical features, a **neck** that fuses features across scales (e.g., FPN, PANet, BiFPN), and a **head** that predicts, for each location of each output feature map, a classification score and a bounding box.

2.6.2 Anchor-based and anchor-free detection

Anchor-based detectors place predefined boxes of several scales and aspect ratios at each location and regress offsets relative to them. Anchor-free detectors such as FCOS and CenterNet predict boxes directly from points. Anchor-free methods have fewer hyper-parameters, fewer outputs and are therefore attractive for lightweight models.

2.6.3 Bounding box regression losses

The Intersection over Union between a predicted box B and a ground-truth box B^{gt} is $IoU = |B \cap B^{gt}| / |B \cup B^{gt}|$. Generalised IoU (GIoU), Distance IoU (DIoU) and Complete IoU (CIoU) losses extend IoU to provide gradients even for non-overlapping boxes. The

CIoU loss is:

$$L_{CIoU} = 1 - IoU + \rho^2(b, b^{gt}) / c^2 + \alpha v \tag{2.13}$$

where ρ is the Euclidean distance between box centres, c is the diagonal length of the smallest enclosing box, v measures aspect-ratio consistency and α is a trade-off weight.

2.6.4 Classification losses

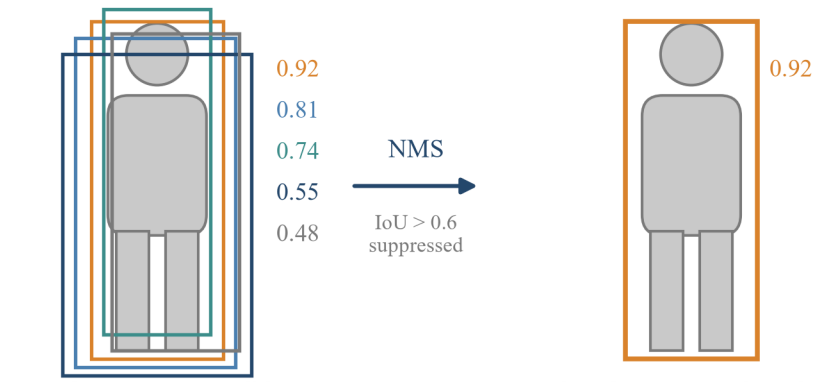
Dense detectors suffer from extreme imbalance between the few positive locations and the many negative ones. Focal Loss down-weights easy examples:

$$FL(p_t) = -\alpha_t (1 - p_t)^{\gamma} \log(p_t) \tag{2.14}$$

where p_t is the predicted probability of the true class and γ (typically 2) controls the focusing strength.

2.6.5 Non-maximum suppression

Because dense detectors produce many overlapping predictions for each object, non-maximum suppression (NMS) keeps the highest-scoring box and removes other boxes whose IoU with it exceeds a threshold. In the single-class setting NMS needs to be run on only one class, which reduces post-processing time.



(a) Before NMS: 5 candidates (b) After NMS: 1 detection

Figure 2.3 Non-maximum suppression: overlapping candidate boxes on one person (left) are reduced to the highest-scoring detection (right). Scores are illustrative.

2.6.6 Variants of non-maximum suppression

Standard (greedy) NMS sorts all detections by score, repeatedly selects the highest-scoring box M, outputs it and removes every remaining box b_i with $IoU(M, b_i) \geq N_t$. Its rescoring rule can be written as $s_i \leftarrow s_i$ if $IoU(M, b_i) < N_t$, and $s_i \leftarrow 0$ otherwise. The hard threshold is problematic in crowded scenes: a true positive that overlaps a neighbouring pedestrian by more than N_t is suppressed entirely.

Soft-NMS [46] replaces suppression with score decay. With the Gaussian penalty function the update is:

$$s_i \leftarrow s_i \cdot exp(-IoU(M, b_i)^2 / \sigma) \quad (2.15)$$

so that heavily overlapping boxes are not removed but ranked lower, which improves recall in crowded scenes at the cost of slightly more computation and an additional final score threshold. DIoU-NMS [19] uses the DIoU criterion instead of IoU, i.e. a box is suppressed only if $IoU(M, b_i) - \rho^2(b_M, b_i) / c^2 \geq N_t$. Two boxes with high overlap but well-separated centres — typical of two people standing one behind the other — are therefore less likely to be merged. Other variants include weighted NMS, which averages the coordinates of suppressed boxes weighted by their scores, and Matrix NMS, which computes the decay of all boxes in parallel using the IoU matrix and is attractive for GPU implementation. Since LSD-Net is deployed on embedded hardware, the cost of each variant is measured together with its accuracy in Chapter 5.

2.7 Evaluation Metrics

- **Precision and Recall**: P = TP / (TP + FP), R = TP / (TP + FN).
- **Average Precision (AP)**: the area under the precision–recall curve. AP@0.5 uses an IoU threshold of 0.5; AP@0.5:0.95 averages over thresholds from 0.5 to 0.95 in steps of 0.05 (COCO protocol).
- **Log-average miss rate (MR^{-2})**: the standard pedestrian detection metric, averaging miss rate over false positives per image in the range $[10^{-2}, 10^{0}]$.
- **Parameters, FLOPs and model size**: measures of model complexity.
- **Latency and FPS**: measured end-to-end, including pre-processing and NMS, on the target hardware.

2.7.1 Computation of average precision

The AP values reported in this thesis follow the COCO protocol [25], which is computed by the following procedure for each IoU threshold τ:

- Collect all detections over the evaluation set and sort them by descending score; keep at most 100 detections per image.
- For each detection in order, find the unmatched ground truth in the same image with the highest IoU. If that IoU is at least τ, the detection is a true positive and the ground truth is marked as matched; otherwise it is a false positive. Detections matched to ground truths marked as "crowd" or "ignore" are neither true nor false positives.
- Compute cumulative TP and FP counts after each detection, and from them the precision $P_k = TP_k / (TP_k + FP_k)$ and recall $R_k = TP_k / N_{gt}$.

- Make the precision monotonic by replacing each value with the maximum precision at any equal or higher recall (the precision envelope).
- Sample the envelope at 101 equally spaced recall levels r ∈ {0, 0.01, ..., 1} and average the interpolated precisions.

$$AP_{\tau} = (1/101)\, \Sigma_{r \in \{0,\ 0.01,\ \ldots,\ 1\}}\, max_{R_k \geq r}\, P_k \tag{2.16}$$

AP@0.5:0.95 is the mean of AP_{τ} over τ ∈ {0.50, 0.55, ..., 0.95}; it rewards precise localisation. PASCAL VOC [70] originally used 11-point interpolation and later the exact area under the envelope; the differences are small but the protocol must be kept fixed across compared models. In the single-class case, AP and mean AP coincide. The log-average miss rate is computed analogously from the miss-rate versus false-positives-per-image curve, sampled at nine points evenly spaced in log space between 10^{-2} and 10^{0}, and averaged geometrically.

2.8 Chapter Summary

This chapter presented the basic theory used in this thesis: convolution and its efficient variants, attention mechanisms, the structure of one-stage detectors, loss functions and evaluation metrics. These foundations support the network design in Chapter 3.

Chapter 3
Design of the Lightweight Single-Class Detection Network

3.1 Design Principles

The design of LSD-Net follows five principles derived from the analysis in Chapters 1 and 2 and from the practical guidelines of ShuffleNetV2:

- **Remove multi-class redundancy.** Replace the C-way classification branch with a single objectness/quality score.
- **Minimise memory access cost.** Keep input and output channel widths equal where possible and avoid excessive fragmentation of the network.
- **Spend computation where it matters.** Allocate more capacity to the resolutions at which the target objects typically appear.
- **Prefer hardware-friendly operators.** Use 3 x 3 depthwise and 1 x 1 pointwise convolutions, BN folded into convolutions, and ReLU6 or h-swish activations, all of which quantise well.
- **Anchor-free prediction.** Avoid anchor hyper-parameters and reduce the number of outputs.

3.2 Overall Architecture

The overall architecture of LSD-Net is shown in Figure 3.1. An input image of 416 x 416 pixels (configurable to 320 x 320 for the smallest variant) is processed by the backbone, which produces feature maps C3, C4 and C5 at strides 8, 16 and 32. The neck fuses them into P3, P4 and P5 with a uniform reduced channel width. A shared detection head predicts, at every location of each pyramid level, an objectness score and four box distances.

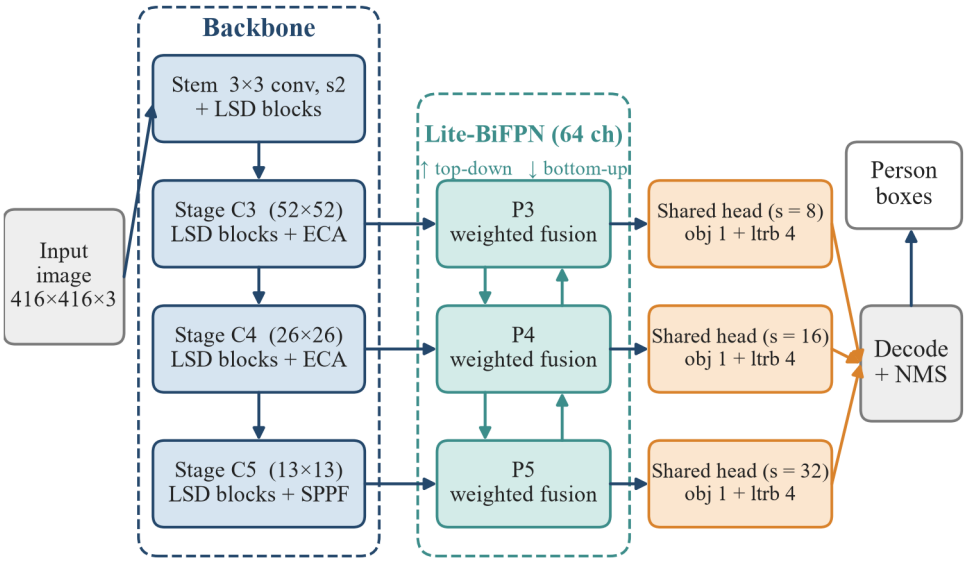


Figure 3.1 Overall pipeline of LSD-Net: a 416×416 input passes through the LSD-block backbone (stages C3/C4/C5, SPPF), the 64-channel Lite-BiFPN neck, shared anchor-free heads at strides 8/16/32, and decoding with NMS.

3.3 Lightweight Backbone

3.3.1 Stem

The stem consists of a 3 x 3 standard convolution with stride 2 and 16 output channels followed by a 3 x 3 depthwise separable convolution. Using a standard convolution at the first layer is efficient because the number of input channels is only three.

3.3.2 LSD block

The basic building unit is the LSD block (Figure 3.2), an inverted residual block augmented with ECA attention:

- 1 x 1 pointwise convolution expanding channels by factor t (t = 3 or 4), followed by BN and h-swish;
- 3 x 3 (or 5 x 5 in deeper stages) depthwise convolution with stride s, followed by BN and h-swish;
- ECA channel attention;
- 1 x 1 linear pointwise projection followed by BN;
- residual connection when s = 1 and input and output channels match.

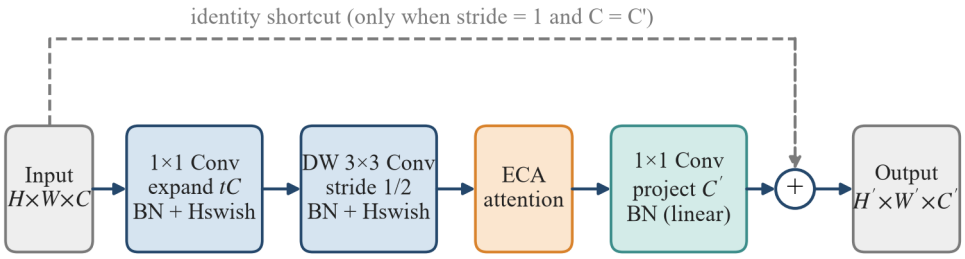


Figure 3.2 Structure of the LSD block: 1×1 expansion, 3×3 depthwise convolution, ECA channel attention and linear 1×1 projection, with an identity shortcut when stride = 1 and input/output channels match.

3.3.3 Backbone configuration

The backbone configuration is given in Table 3.1. Its width can be scaled by a multiplier w ∈ {0.5, 0.75, 1.0} to produce the LSD-Net-S, LSD-Net-M and LSD-Net-L variants.

Stage	Output size	Operator	t	Channels	Repeats	Stride
Stem	208 x 208	Conv 3x3 + DWSep	–	16	1	2
Stage 1	104 x 104	LSD block 3x3	3	24	2	2
Stage 2 (C3)	52 x 52	LSD block 3x3	3	40	3	2
Stage 3 (C4)	26 x 26	LSD block 5x5	4	80	4	2
Stage 4 (C5)	13 x 13	LSD block 5x5	4	160	2	2
SPPF	13 x 13	Fast spatial pyramid pooling	–	160	1	1

Table 3.1 Backbone configuration of LSD-Net (width multiplier 1.0, 416 x 416 input).

A lightweight Spatial Pyramid Pooling – Fast (SPPF) module is attached to the last stage. It applies three successive 5 x 5 max-pooling operations and concatenates their outputs, enlarging the receptive field at negligible cost. This is important for single-class pedestrian detection because pedestrians are tall objects that require vertical context.

3.3.4 Layer-by-layer design rationale

Stem. The first layer operates on the full-resolution image and therefore has the largest spatial extent of the network. A depthwise convolution on three input channels would save almost nothing, whereas a standard 3 x 3 convolution with stride 2 costs 208 · 208 ·

$3 \cdot 16 \cdot 9 \approx 18.7$ M multiply–accumulate operations (MACs) at 416 x 416, a small fraction of the total. Reducing resolution immediately by a factor of two is the single most effective way to reduce cost in the early network, and 16 channels are sufficient to represent edges and colour blobs at this level.

Stage 1 (stride 4). Features at stride 4 are not used for prediction, so stage 1 only needs to provide good low-level features to later stages. Only two blocks with a small expansion factor ($t = 3$) are used, because the cost of each block scales with the spatial area, which is largest here.

Stage 2 (stride 8, C3). C3 is the finest level used by the head and is responsible for small objects, such as distant pedestrians whose height is only 16–48 pixels at the input resolution. Three blocks are used to provide adequate semantic depth to this level. The 3 x 3 kernel is retained because the receptive field already greatly exceeds the size of the smallest objects (see the receptive-field analysis below).

Stage 3 (stride 16, C4). This stage contains the most blocks (four) because it has a moderate spatial size, so depth is cheap, and because medium-sized pedestrians constitute the majority of instances in the target datasets. The expansion factor is increased to $t = 4$ and the kernel to 5 x 5. A 5 x 5 depthwise kernel costs $25/9 \approx 2.8$ times a 3 x 3 depthwise kernel, but since depthwise convolutions account for only a small proportion of block cost (the pointwise layers dominate), the larger kernel increases total block FLOPs only modestly while expanding the receptive field substantially.

Stage 4 (stride 32, C5) and SPPF. The deepest stage handles large, near-camera pedestrians and supplies global context to the neck. The SPPF module enlarges the effective context to cover most of the image at the cost of only max-pooling and two 1 x 1 convolutions.

Placement of ECA. ECA is placed after the depthwise convolution, where the expanded representation contains spatially filtered features of all channels, and before the linear projection, so that attention can suppress channels before they are mixed. ECA is preferred over SE because SE adds $2C^2/r$ parameters per block, which for $t \cdot C = 640$ expanded channels and reduction ratio $r = 4$ would exceed the parameters of the depthwise convolution itself. The adaptive kernel size of ECA [18] is $k = |\log_2(C)/\gamma + b/\gamma|_{odd}$ with $\gamma = 2$ and $b = 1$, giving $k = 3$ or 5 for the channel widths of LSD-Net.

Choice of activation. h-swish is used in the backbone because it gives accuracy close to Swish [12] while remaining piecewise linear. In the first stage, where the activation is applied to the largest tensors, ReLU is evaluated as a cheaper alternative in the ablation study.

3.3.5 Receptive-field analysis

For a stack of layers indexed by i with kernel size k_i and stride s_i, the theoretical receptive field r_L of a unit in layer L grows according to the recursion

$$r_i = r_{i-1} + (k_i - 1) \cdot j_{i-1}, j_i = j_{i-1} \cdot s_i, r_0 = 1, j_0 = 1 \quad (3.1)$$

where j_i is the cumulative stride ("jump") of layer i. 1 x 1 convolutions do not enlarge the receptive field. Applying the recursion to the configuration of Table 3.1, considering only the spatial (3 x 3, 5 x 5 and pooling) layers, gives the theoretical receptive fields listed in Table 3.2. The values follow directly from the recursion and the configuration; they should be re-checked if the configuration is modified.

Feature	**Stride**	**Spatial layers up to this point**	**Theoretical receptive field (pixels)**
Stem output	2	Conv3x3/s2, DW3x3/s1	7
Stage 1	4	+ 2 x DW3x3 (first s2)	19
C3	8	+ 3 x DW3x3 (first s2)	59
C4	16	+ 4 x DW5x5 (first s2)	283
C5	32	+ 2 x DW5x5 (first s2)	475
C5 + SPPF	32	+ three 5x5 max-pool (equiv. 5, 9, 13)	859 (largest branch)

Table 3.2 Theoretical receptive field of backbone outputs of LSD-Net (416 x 416 input).

The theoretical receptive field overestimates the region that actually influences a unit. Luo et al. [74] showed that the effective receptive field has an approximately Gaussian shape and occupies only a fraction of the theoretical one, growing with the square root of depth. The design implication is that the theoretical receptive field at each level should comfortably exceed the size of the largest objects assigned to it. Although the dynamic assignment does not impose explicit size ranges, its centre prior and IoU cost in practice assign small objects mainly to P3; for a pedestrian up to roughly 128 pixels tall, the 59-pixel field of C3 alone would be insufficient, but after top-down fusion with C4 (283 pixels) in the neck, P3 features see sufficient context; objects on P5 can span the whole image, which is covered by the SPPF-enhanced receptive field. Pedestrians are tall, with a typical height-to-width ratio of about 2.4 to 3 for full-body boxes, so vertical context is especially important; the large 5 x 5 kernels in stages 3 and 4 and the SPPF module were

chosen with this in mind.

3.4 Lightweight Feature Fusion Neck

Objects of a single class still appear at very different scales. A feature pyramid is therefore retained, but in a lightweight form called Lite-BiFPN:

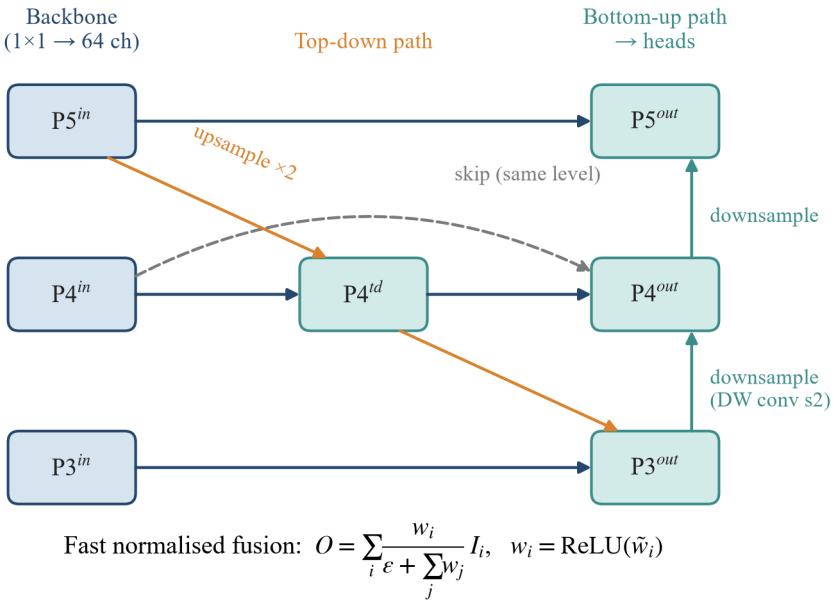


Figure 3.3 Lite-BiFPN with top-down and bottom-up paths over P3–P5, same-level skip connection and fast normalised weighted fusion at each node.

- All inputs are first projected to a common width of 64 channels by 1 x 1 convolutions (96 for LSD-Net-L).
- A top-down path upsamples coarse features and fuses them with finer features; a bottom-up path then propagates fine localisation detail back to coarse levels.
- Fusion uses fast normalised weighted addition rather than concatenation, which keeps the channel width constant and reduces memory traffic:

$$O = \Sigma_i \left(w_i / (\varepsilon + \Sigma_j w_j) \right) \cdot I_i,\ w_i \geq 0 \qquad (3.2)$$

- After each fusion, a single 3 x 3 depthwise separable convolution refines the features.

Compared with PANet with concatenation used by YOLOv5, Lite-BiFPN reduces neck FLOPs considerably while retaining bidirectional information flow.

3.4.1 Fusion order and neck topology

The Lite-BiFPN contains one top-down and one bottom-up pass, five fusion nodes in total. The top-down nodes are $P4^{td}$ = DWSep($w_1 \cdot P4^{in} + w_2 \cdot \mathrm{Up}(P5^{in})$) and $P3^{out}$ = DWSep($w_1 \cdot P3^{in} + w_2 \cdot \mathrm{Up}(P4^{td})$). The bottom-up nodes are $P4^{out}$ = DWSep($w_1 \cdot P4^{in} + w_2 \cdot P4^{td} + w_3 \cdot \mathrm{Down}(P3^{out})$) and $P5^{out}$ = DWSep($w_1 \cdot P5^{in} + w_2 \cdot \mathrm{Down}(P4^{out})$), where

Up is nearest-neighbour upsampling by two and Down is a 3 x 3 depthwise convolution with stride 2. The extra skip input to $P4^{out}$ follows BiFPN [16] and gives the middle level, which carries most pedestrians, access to both unrefined and top-down-refined features. The fusion weights are passed through ReLU and normalised by the fast normalised fusion rule given above; after training they become constants and are folded into the preceding 1 x 1 projection or into a per-input scale, so they cost nothing at inference.

Unlike EfficientDet, the BiFPN layer is not repeated. Repetition multiplies the neck cost, and preliminary analysis suggests that in a single-class setting the marginal benefit of a second pass is small compared with spending the same budget on backbone depth; this is verified by the neck-depth ablation in Chapter 5.

3.5 Anchor-Free Single-Class Detection Head

3.5.1 Head structure

Each pyramid level shares one head consisting of two 3 x 3 depthwise separable convolutions followed by two output branches: an **objectness branch** with one output channel and a **regression branch** with four output channels representing distances (l, t, r, b) from the location to the four sides of the box. Sharing weights across levels reduces parameters; a learnable scalar per level rescales the regression output to account for different object sizes.

3.5.2 Single-class objectness as quality estimation

In a multi-class detector, the classification branch outputs C scores. In LSD-Net, a single score is predicted and trained to estimate the IoU between the predicted box and the matched ground truth (an IoU-aware quality score, similar in spirit to Varifocal and Quality Focal Loss). This unifies classification and localisation quality, so that the score used in NMS ranks boxes by their expected localisation accuracy, which improves AP at high IoU thresholds.

3.5.3 Why one score suffices

In a C-class detector with a sigmoid head, the classification output at a location is a C-dimensional vector and the score of class c is used for ranking within class c. When C = 1, there is no inter-class competition, so the score has a single role: to rank all candidate boxes such that the ranking agrees with their true localisation quality, which is precisely what AP measures. A separate centre-ness or IoU branch, as in FCOS [8], multiplies two independently trained scores, and the product is not explicitly trained; using a single IoU-aware target unifies the training and inference representations [20] and removes one output channel and its convolution. The head thus outputs 5 channels

per location instead of the 4 + C (+1) channels of a generic anchor-free head, and its two convolutional layers are shared by the objectness and regression branches rather than duplicated in a decoupled head. The decoupled head of YOLOX [21] was motivated by conflicts between classification and regression; with only one class, the classification sub-task is much simpler, and sharing the trunk halves the head cost. The impact of this choice is measured by the head ablation in Chapter 5.

3.5.4 Box decoding

For a location (x, y) on a level with stride s, the predicted box is:

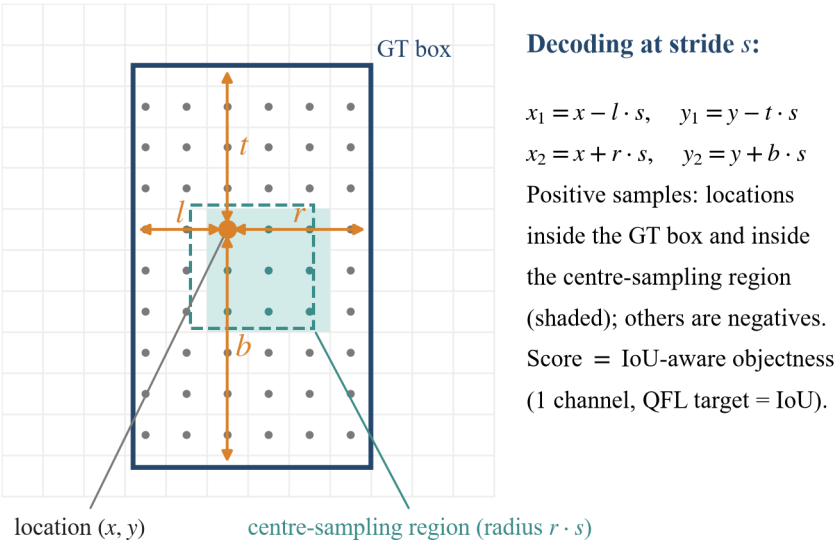


Figure 3.4 Anchor-free box encoding: a feature-map location predicts distances (l, t, r, b) to the box edges; only locations inside the centre-sampling region of the ground-truth box are positive samples.

$$x_1 = x - s{\cdot}l,\ y_1 = y - s{\cdot}t,\ x_2 = x + s{\cdot}r,\ y_2 = y + s{\cdot}b \tag{3.3}$$

3.6 Loss Function

The total loss is:

$$L = \lambda_{obj} L_{QFL} + \lambda_{box} L_{CIoU} + \lambda_{kd} L_{KD} \tag{3.4}$$

where L_{QFL} is the quality focal loss on the objectness score, L_{CIoU} is the CIoU box loss computed on positive samples, and L_{KD} is the distillation loss described in Chapter 4. Default weights are $\lambda_{obj} = 1.0$, $\lambda_{box} = 2.0$ and $\lambda_{kd} = 1.0$. The quality focal loss is:

$$L_{QFL}(\sigma) = -|y - \sigma|^{\beta} \left[(1 - y) \log(1 - \sigma) + y \log(\sigma) \right] \tag{3.5}$$

where y is the IoU target (0 for negatives), σ is the predicted score and $\beta = 2$.

3.6.1 Derivation of the CIoU loss

Let the predicted box have centre b = (x, y), width w and height h, and the ground truth have centre b^{gt}, width w^{gt} and height h^{gt}. The plain IoU loss 1 − IoU has zero gradient when the boxes do not overlap and does not distinguish between different relative placements with the same IoU. GIoU [45] adds the penalty $|C \setminus (B \cup B^{gt})| / |C|$, where C is the smallest enclosing box, but when one box contains the other the penalty vanishes and GIoU degenerates to IoU, so convergence is slow. DIoU [19] instead directly minimises the normalised distance between centres:

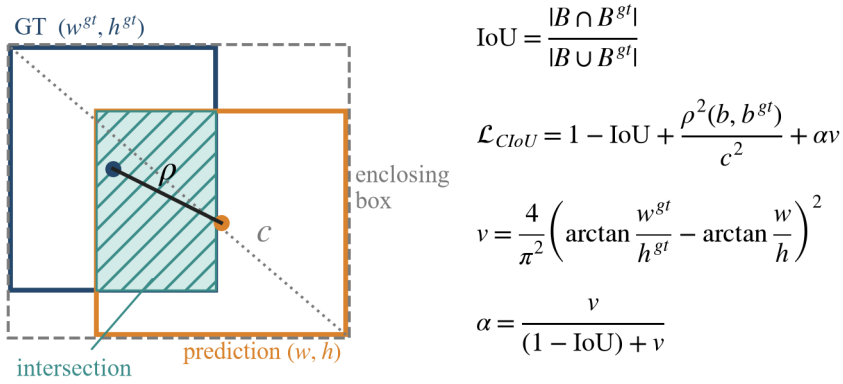


Figure 3.5 Geometry of the CIoU loss: intersection of predicted and ground-truth boxes, centre distance ρ and diagonal c of the smallest enclosing box.

$$L_{DIoU} = 1 - IoU + \rho^2(b, b^{gt}) / c^2 \tag{3.6}$$

where c is the diagonal length of C. CIoU adds an aspect-ratio consistency term:

$$v = (4 / \pi^2)\,(\arctan(w^{gt} / h^{gt}) - \arctan(w / h))^2,\ \alpha = v / ((1 - IoU) + v) \tag{3.7}$$

The factor $4/\pi^2$ normalises v to [0, 1], because the difference of two arctangents of positive ratios lies in $(-\pi/2, \pi/2)$. The trade-off α is treated as a constant during back-propagation; it gives higher priority to the overlap term when IoU is small and to the aspect-ratio term once the boxes overlap well. The gradient of v with respect to w is

$$\partial v/\partial w = (8 / \pi^2)\,(\arctan(w^{gt}/h^{gt}) - \arctan(w/h)) \cdot h / (w^2 + h^2) \tag{3.8}$$

and $\partial v/\partial h = -(w/h) \cdot \partial v/\partial w$. For small boxes the denominator $w^2 + h^2$ becomes small and the gradient can explode; following [19], the factor $1/(w^2 + h^2)$ is replaced by 1 during back-propagation, which preserves the gradient direction. For pedestrians, whose aspect ratio is relatively stable, the aspect-ratio term mainly accelerates early training; its contribution is isolated in the loss ablation.

Because the head predicts distances (l, t, r, b), the box is a differentiable function of the outputs through the decoding equations above, and the CIoU gradient flows to all four distances. Distances are predicted as $\exp(s_L \cdot u)$ or $\mathrm{ReLU}(s_L \cdot u)$, where u is the raw output and s_L the learnable scale of level L; ReLU is preferred for INT8 deployment

because the exponential is costly and amplifies quantisation errors.

3.6.2 Derivation of the quality focal loss

Let $\sigma \in (0, 1)$ be the predicted score and $y \in [0, 1]$ the soft target, equal to the IoU between the decoded prediction and its assigned ground truth for positives and 0 for negatives. The binary cross-entropy with a soft target, $-[(1 - y) \log(1 - \sigma) + y \log \sigma]$, is minimised at $\sigma = y$, so the score learns to estimate IoU. The focal modulating factor of [6] is defined for hard labels as $(1 - p_t)^{\gamma}$; its continuous generalisation is $|y - \sigma|^{\beta}$, which tends to zero as the estimate approaches the target and therefore down-weights easy examples of both kinds — including the large number of easy background locations. The gradient with respect to the logit z ($\sigma = \text{sigmoid}(z)$) for $\beta = 2$ is

$$\partial L_{QFL}/\partial z = |y - \sigma|^2 (\sigma - y) - 2 |y - \sigma| \cdot sign(\sigma - y) \cdot \sigma(1 - \sigma) \cdot [(1 - y) \log(1 - \sigma) + y \log \sigma] \quad (3.9)$$

whose sign always drives σ towards y, so the global minimum remains at $\sigma = y$ [20]. The loss is normalised by the number of positive samples, or by the sum of their IoU targets to reduce noise when there are few positives in an image. The IoU target is computed with the predicted box and detached from the graph so that the score does not influence box regression through the target.

3.6.3 Loss weighting of positives

The CIoU loss of each positive sample is weighted by its IoU target, so that well-localised positives — which have high quality scores at inference time and thus dominate AP — receive more localisation gradient, while ambiguous positives on object borders receive less. The total box loss is

$$L_{box} = \Sigma_{i \in Pos} y_i \cdot L_{CIoU}(B_i, B_i^{gt}) / \Sigma_{i \in Pos} y_i \quad (3.10)$$

3.7 Label Assignment

Label assignment decides which locations are trained as positives for each ground-truth box. LSD-Net uses a dynamic assignment strategy inspired by SimOTA: candidate locations are first restricted to the centre region of each ground-truth box (centre sampling with radius 2.5 strides), then for each ground truth the top-k candidates by a cost combining objectness and IoU are selected, with k determined dynamically from the sum of the top IoUs. This produces a variable number of positives per object, adapting to object size and difficulty, and is particularly beneficial in crowded pedestrian scenes where boxes overlap heavily.

3.7.1 Step-by-step assignment algorithm

The dynamic assignment is computed independently for each training image, with no gradients, at every iteration. Let G be the set of ground-truth boxes and A the set of all locations on P3, P4 and P5 (3 549 locations at 416 x 416: $52^2 + 26^2 + 13^2$).

- **Step 1 — geometric prior.** For each location a and ground truth g, mark a as a candidate of g if a lies inside g *or* within a square of side $2 \cdot 2.5 \cdot s_a$ centred at the centre of g, where s_a is the stride of a. Locations that are candidates of no ground truth are negatives and are excluded from the remaining steps.
- **Step 2 — pairwise cost.** For every candidate pair (a, g), decode the current prediction of a and compute IoU_{ag} and the cost $C_{ag} = BCE(\sigma_a, IoU_{ag}) + \lambda_c \cdot (-\log(IoU_{ag} + \varepsilon)) + 10^5 \cdot$ [a not inside g and not in centre region of g], with $\lambda_c = 3$. The large constant effectively forbids candidates that satisfy neither geometric condition.
- **Step 3 — dynamic k.** For each ground truth g, take the q = min(10, |candidates|) largest IoU_{ag} values and set k_g = max(1, floor(Σ top-q IoU)). Large, well-predicted objects receive more positives; small or currently poorly predicted objects receive at least one.
- **Step 4 — selection.** For each g, select the k_g candidates with the lowest cost C_{ag} as positives of g.
- **Step 5 — conflict resolution.** If a location is selected by more than one ground truth, keep only the ground truth with the lowest cost for that location. In crowded scenes this step prevents a single location from receiving contradictory regression targets.
- **Step 6 — targets.** For every positive a assigned to g, the regression target is the distances from a to the four sides of g divided by the stride, and the quality target is IoU_{ag} computed from the current (detached) prediction. All other locations have quality target 0. Locations inside "ignore" regions of CrowdHuman receive zero loss weight.

At the beginning of training the predictions are random and the IoU-based cost is uninformative, so the geometric prior dominates and the assignment behaves like centre sampling. As training proceeds the cost increasingly reflects prediction quality, producing a curriculum from geometric to prediction-aware assignment. The cost of the assignment is O(|G| · |candidates|) per image and is negligible compared with the forward and backward passes. Unlike full optimal-transport assignment, which solves a Sinkhorn iteration over all pairs, this procedure requires only a top-k selection per ground truth, as in SimOTA [21].

3.8 Complexity Analysis

3.8.1 Per-layer FLOP and parameter formulas

The complexity of LSD-Net can be derived analytically, layer by layer. Throughout, one multiply–accumulate operation (MAC) is counted as one FLOP, which is the convention of most detection papers; the profiling tool must be configured accordingly. For an output feature map of size H_o x W_o the formulas are:

- Standard convolution k x k, $C_{in} \rightarrow C_{out}$: Params = $k^2 C_{in} C_{out}$ (+ C_{out} bias); FLOPs = $H_o W_o k^2 C_{in} C_{out}$.
- Depthwise convolution k x k on C channels: Params = $k^2 C$; FLOPs = $H_o W_o k^2 C$.
- Pointwise convolution $C_{in} \rightarrow C_{out}$: Params = $C_{in} C_{out}$; FLOPs = $H_o W_o C_{in} C_{out}$.
- BN: 2C parameters at training time (γ, β), zero after folding.
- ECA: k parameters; FLOPs ≈ HWC (pooling) + kC (1-D convolution) + HWC (re-weighting).
- Weighted fusion of n inputs: n parameters; FLOPs ≈ n·HWC.

An LSD block with input width C_{in}, output width C_{out}, expansion t, kernel k and stride s, operating on an input of size H x W, therefore costs

$$FLOPs_{LSD} = H W \cdot t C_{in}^{\ 2} + (H W / s^2) \cdot (k^2 t C_{in} + t C_{in} C_{out}) \quad (3.11)$$

$$Params_{LSD} = t C_{in}^{\ 2} + k^2 t C_{in} + k_{eca} + t C_{in} C_{out} \quad (3.12)$$

For a typical stride-1 block in stage 3 (H = W = 26, $C_{in} = C_{out}$ = 80, t = 4, k = 5), the expansion and projection layers each account for 26 · 26 · 320 · 80 ≈ 17.3 M FLOPs, whereas the 5 x 5 depthwise layer accounts for 26 · 26 · 25 · 320 ≈ 5.4 M FLOPs. More than three quarters of the block cost lies in the pointwise layers, which justifies the use of the larger depthwise kernel and identifies the expansion factor t as the main lever for controlling cost.

For the neck with uniform width C_n, each fusion node costs approximately H W (9 C_n + $C_n^{\ 2}$) for its depthwise separable refinement, plus the 1 x 1 input projections C_i C_n H_i W_i for each backbone output. For a concatenation-based PAN node, the subsequent convolution has $2C_n$ input channels, which doubles the pointwise cost; this is the analytical source of the saving of Lite-BiFPN. The head, shared across levels, costs Σ_L H_L W_L · [2 (9 C_n + $C_n^{\ 2}$) + 5 C_n] FLOPs. Because 52^2 is 16 times 13^2, the P3 level dominates head and neck cost, which motivates keeping the head shallow.

Width scaling by multiplier w multiplies every pointwise term by approximately w^2 and every depthwise term by w, so total FLOPs scale between w and w^2; changing the input from 416 to 320 multiplies all terms by $(320/416)^2 \approx 0.59$. These relations allow the expected complexity of each variant to be checked against the profiler output in Table

3.3.

Table 3.3 lists the theoretical complexity of the three variants. Values are computed analytically from the architecture and should be confirmed by the profiling tool used in the experiments.

Variant	Width	Input	Params (M)	FLOPs (G)
LSD-Net-S	0.50	320 x 320	[measure]	[measure]
LSD-Net-M	0.75	416 x 416	[measure]	[measure]
LSD-Net-L	1.00	416 x 416	[measure]	[measure]

Table 3.3 Complexity of the LSD-Net variants (to be filled from profiling, e.g. with thop or fvcore).

3.9 Design Alternatives Considered

Several alternatives were considered during the design and rejected for the reasons summarised below. Where the reason is empirical, the corresponding experiment is included in Chapter 5 so that the decision can be verified.

- **Anchor-based head.** Anchors tailored to pedestrian aspect ratios could encode a useful prior, but they multiply the number of outputs by the number of anchors, require dataset-specific clustering and complicate assignment. Anchor-free prediction with dynamic assignment learns the aspect-ratio prior implicitly.
- **CenterNet-style single heatmap at stride 4.** Such a design avoids NMS, but a stride-4 output at 416 x 416 has 10 816 positions per channel and requires upsampling layers whose cost exceeds that of the whole LSD-Net neck. Moreover, centre-point heatmaps struggle when the centres of occluding pedestrians nearly coincide.
- **RepVGG-style re-parameterised blocks.** Plain 3 x 3 dense convolutions are fast on desktop GPUs but have a much larger FLOP and parameter count than depthwise separable blocks at the same width; on ARM CPUs and low-power accelerators the latter are more efficient. Re-parameterisation of the depthwise layer (parallel 3 x 3 and 1 x 1 depthwise branches merged at inference) remains a compatible option and is tested as an ablation.
- **Transformer or MobileViT blocks in the last stage.** A global attention block on the 13 x 13 map is affordable in FLOPs, but softmax and layer normalisation are poorly supported by INT8 toolchains on the target devices; SPPF provides large context with operators that quantise exactly.
- **SE attention or CBAM.** Rejected because of parameter cost (SE) and the additional spatial pooling and convolution layers (CBAM), which add latency disproportionate

to their FLOPs.

- **Concatenation-based PAN neck.** Rejected on the grounds of the analytical cost argument above; retained as the baseline in the ablation.
- **Decoupled head.** Rejected for the single-class case because the classification sub-task is simple; evaluated in the head ablation.
- **Four pyramid levels (adding P6 or P2).** P2 would help very small pedestrians but quadruples the cost of the finest level; P6 is unnecessary at 416 x 416 because P5 already covers the full image.

3.10 Summary of Architecture Hyper-Parameters

Component	Hyper-parameter	Value (LSD-Net-M)
Backbone	width multiplier	0.75
Backbone	expansion factors per stage	3, 3, 4, 4
Backbone	depthwise kernels per stage	3, 3, 5, 5
Backbone	activation	h-swish (ReLU option in stage 1)
ECA	kernel size	adaptive, 3 or 5
SPPF	pooling kernel	5 (three times)
Neck	width C_n	64
Neck	number of BiFPN passes	1
Head	depthwise separable layers	2, shared across levels
Head	outputs per location	1 quality + 4 distances
Assignment	centre radius / top-q / λ_c	2.5 strides / 10 / 3.0
Post-processing	score threshold / NMS IoU	0.3 / 0.6

Table 3.4 Architecture hyper-parameters of LSD-Net-M.

3.11 Chapter Summary

This chapter presented the design of LSD-Net. The backbone uses LSD blocks with ECA attention; the neck uses Lite-BiFPN with weighted fusion; the head is anchor-free, shared across levels, and predicts a single IoU-aware quality score. The design removes redundant multi-class components and favours hardware-friendly operators.

Chapter 4
Training, Distillation and Quantisation Strategy

4.1 Dataset Construction

4.1.1 Data sources

Pedestrians are chosen as the representative single class because of their practical importance and because pedestrian detection is challenging (large scale variation, occlusion, crowding). The dataset is constructed from:

- **CrowdHuman**: about 15,000 training and 4,370 validation images with full-body, visible-body and head annotations; full-body boxes are used.
- **COCO-person**: images of the 2017 split containing the "person" category, with all other categories discarded.
- Optionally, a self-collected dataset from the target deployment scene, annotated in YOLO format with LabelImg or CVAT.

4.1.2 Pre-processing

All annotations are converted to a unified format (class index 0, normalised centre coordinates and size). Boxes smaller than 4 pixels in either dimension at the training resolution are removed or marked as ignore regions. Images are letter-boxed to the network input size to preserve aspect ratio.

4.2 Data Augmentation

- **Mosaic**: combines four images into one, increasing object density and scale diversity. It is disabled during the last 15 epochs so that the model adapts to the natural image distribution.
- **MixUp** with low probability for the larger variant only; small networks are under-fitted and benefit less from strong regularisation.
- **Random affine**: scaling (0.5–1.5), translation and horizontal flip.
- **HSV colour jitter**: hue, saturation and value perturbations for robustness to illumination.

4.3 Training Settings

Item	Setting
Framework	PyTorch 2.x, CUDA

Item	Setting
Optimiser	SGD, momentum 0.937, weight decay 5e-4 (no decay on BN and bias)
Learning rate	0.01 initial, 3-epoch linear warm-up, cosine decay to 0.0005
Batch size	64 (adjust to GPU memory)
Epochs	300
EMA	Exponential moving average of weights, decay 0.9998
Mixed precision	Enabled (FP16 automatic mixed precision)

Table 4.1 Training hyper-parameters.

4.4 Knowledge Distillation

Small networks suffer from limited capacity. To narrow the gap to a larger model, knowledge distillation is applied with a stronger teacher — for example a YOLOv8-m model fine-tuned on the same single-class dataset. Two distillation terms are combined:

- **Feature distillation** on neck outputs, using an adaptation layer (1 x 1 convolution) to match the student channels to the teacher and a foreground mask so that the loss focuses on object regions rather than the dominant background:

$$L_{feat} = (1 / N_p) \Sigma M_{ij} || \varphi(F^S_{ij}) - F^T_{ij} ||^2 \quad (4.1)$$

- **Response distillation** on the objectness score, using the binary cross-entropy between the student and teacher scores with temperature T.

The total distillation loss is $L_{KD} = L_{feat} + L_{resp}$. The teacher is frozen and runs only during training, so distillation adds no inference cost.

4.4.1 Teacher selection and preparation

The teacher must be accurate on the target single-class task and must produce outputs that are structurally compatible with those of the student. Two options are considered: (a) a large anchor-free model of the same family as the student, i.e. an LSD-style head attached to a wider and deeper backbone, which makes response distillation exact; and (b) an off-the-shelf detector such as YOLOv8-m retrained on the single-class dataset, whose objectness-free, class-score output is mapped to a single score. Option (a) guarantees matching pyramid strides and score semantics; option (b) is easier to obtain. The teacher is trained to convergence with the same augmentation and assignment, and its EMA weights are frozen before distillation.

4.4.2 Feature distillation details

Distillation is applied on the three neck outputs P3, P4 and P5. The adaptation layer φ is a 1 x 1 convolution followed by BN that maps the student width C_n to the teacher width; it is used only during training and discarded afterwards. Teacher features are standardised per channel (zero mean, unit variance over the spatial dimensions) before the loss is computed, so that the loss is insensitive to the scale of the teacher activations, which can differ by orders of magnitude between levels.

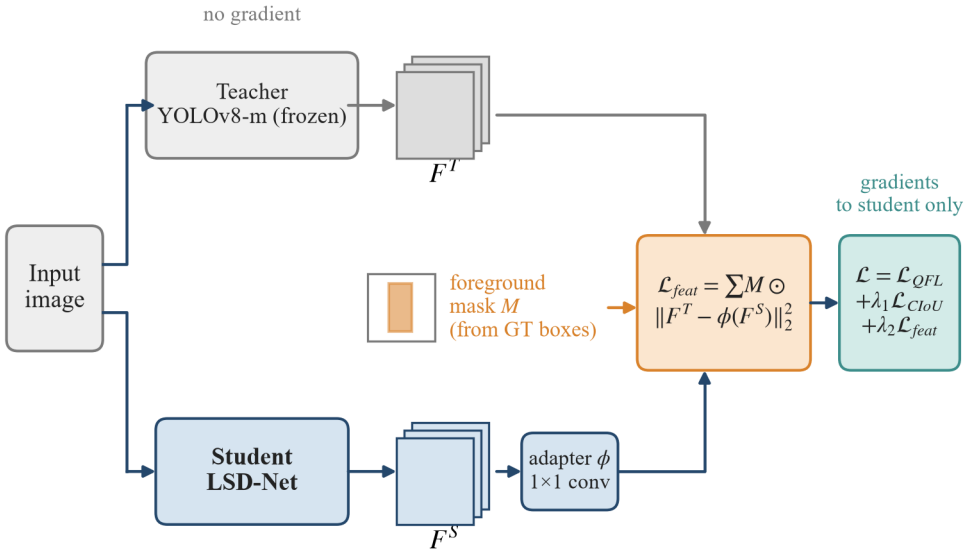


Figure 4.1 Teacher–student feature distillation: the frozen YOLOv8-m teacher guides LSD-Net through a foreground-masked feature-imitation loss added to the detection losses.

The foreground mask M is built from the ground-truth boxes projected onto each level: M_{ij} = 1 for locations inside any ground-truth box (enlarged by one stride), and 0 elsewhere. Because background still contains information about what is *not* a pedestrian, a background term with a smaller weight is added:

$$L_{feat} = \Sigma_L \left[(\alpha_{fg} / N_{fg}) \Sigma M_{ij} D_{ij} + (\alpha_{bg} / N_{bg}) \Sigma (1 - M_{ij}) D_{ij} \right], D_{ij} = \| \varphi(F^S_{ij}) - F^T_{ij} \|^2 \quad (4.2)$$

with α_{fg} = 1.0 and α_{bg} = 0.25 by default, and N_{fg}, N_{bg} the numbers of foreground and background locations. This decomposition follows the observation of [56] that unmasked imitation is dominated by background.

4.4.3 Response and localisation distillation details

The response term uses the teacher's quality score as a soft target for the student at all locations. With temperature T, logits z^S and z^T, the loss is a temperature-scaled binary cross-entropy:

$$L_{resp} = (T^2 / N) \Sigma_a BCE(sigmoid(z^S_a / T), sigmoid(z^T_a / T)) \quad (4.3)$$

The factor T^2 compensates for the reduction of gradient magnitude caused by the temperature, as in [22]. Since the teacher score estimates IoU, this term transfers the teacher's ranking of candidate boxes, including its knowledge of which background regions are confusing.

Optionally, a localisation term transfers box knowledge. Because the LSD-Net head regresses distances directly rather than distributions, the distribution-based localisation distillation of [57] is not applicable without changing the head; instead, the CIoU loss between the student box and the teacher box is applied at locations where the teacher's score exceeds 0.5, weighted by that score. This term is evaluated in the distillation ablation.

4.4.4 Distillation schedule

Distillation losses are enabled after the warm-up period and their weight λ_{kd} is linearly ramped up over the first 10 epochs, because the randomly initialised adaptation layers otherwise produce large early gradients. Distillation is switched off together with mosaic in the last 15 epochs so that the final epochs fine-tune the student on the ground truth alone. Teacher inference is performed on the same augmented batch and doubles the forward cost of training; the training time overhead is reported in Chapter 5.

4.5 Post-Training Quantisation

For deployment, the trained FP32 model is converted to INT8. The procedure is:

- Fold BN layers into the preceding convolutions.
- Export to ONNX with a fixed input size.
- Calibrate activation ranges with 500–1000 representative training images, using entropy (KL-divergence) calibration.
- Build an INT8 TensorRT engine for NVIDIA devices (or use ONNX Runtime / NCNN for CPU and ARM devices).

4.5.1 Quantisation mathematics

Affine (asymmetric) uniform quantisation [23] maps a real value r to an integer q through a scale S and a zero-point Z:

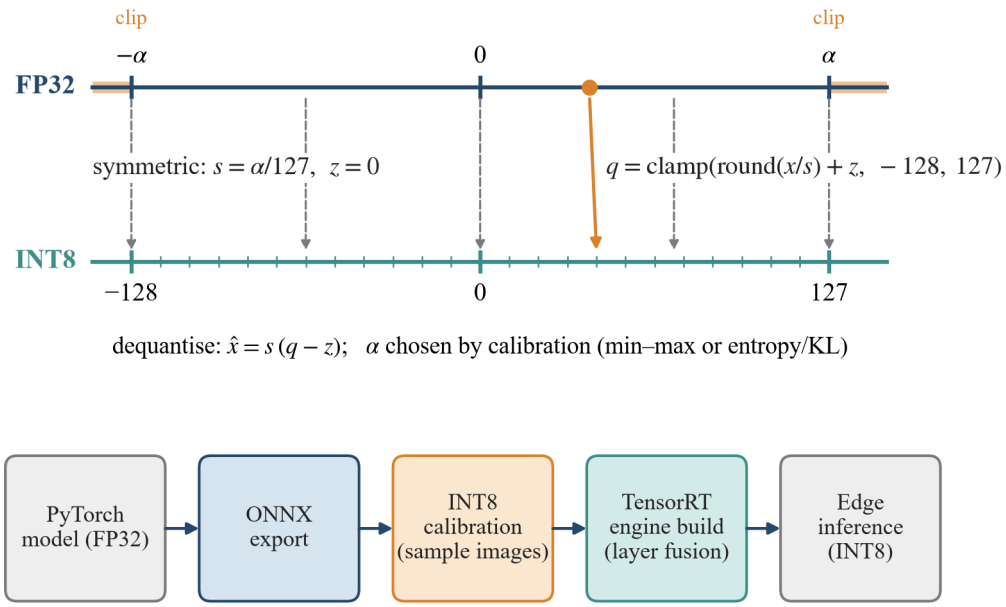


Figure 4.2 Symmetric INT8 quantisation (scale/zero-point mapping from FP32 to INT8 with clipping at ±α) and the deployment flow from PyTorch through ONNX and calibration to a TensorRT INT8 engine.

$$q = clamp(\, round(r / S) + Z,\, q_{min},\, q_{max}\,),\; \hat{r} = S\,(q - Z) \qquad (4.4)$$

For a real range $[r_{min}, r_{max}]$ and an unsigned 8-bit target ($q_{min} = 0$, $q_{max} = 255$), the parameters are $S = (r_{max} - r_{min}) / 255$ and $Z = \text{round}(-r_{min} / S)$; the range is first extended to include zero so that zero is exactly representable, which is necessary for zero padding and ReLU outputs. Symmetric quantisation fixes $Z = 0$ and uses $S = \max(|r_{min}|, |r_{max}|) / 127$ with signed integers; TensorRT uses symmetric quantisation for both weights and activations. The quantisation error of a value inside the range is bounded by $S/2$, whereas values outside the range are clipped, so the choice of range trades rounding error against clipping error.

For a convolution with quantised weights (S_w, $Z_w = 0$) and activations (S_x, Z_x), the output is computed with integer arithmetic as

$$q_y = Z_y + M \cdot \Sigma_k\, q_{w,k}\,(q_{x,k} - Z_x) + q_b,\; M = S_w\, S_x / S_y \qquad (4.5)$$

where the accumulation is performed in 32-bit integers, the bias is quantised with scale $S_w S_x$, and the real multiplier M is implemented as a fixed-point multiplication followed by a bit shift. Per-channel quantisation uses a separate S_w for each output channel; for depthwise convolutions, where each output channel depends on a single filter whose range may differ by orders of magnitude from the others, per-channel weight quantisation is essential [69]. BN folding should be performed before computing weight ranges, because folding rescales each channel by $\gamma / \sqrt{(\sigma^2 + \varepsilon)}$.

4.5.2 Calibration

Weight ranges are known exactly, but activation ranges depend on the input distribution and must be estimated on a calibration set. Three methods are compared:

- **Min–max**: use the observed minimum and maximum. Simple, but a single outlier can inflate S and waste resolution.
- **Percentile**: use, for example, the 99.99th percentile of the absolute values, clipping rare outliers.
- **Entropy (KL divergence)**: build a histogram of activations with fine bins (2048 bins in TensorRT), and for each candidate threshold τ quantise the clipped histogram to 128 levels and compute the Kullback–Leibler divergence between the original distribution P and the quantised distribution Q; choose the τ minimising $KL(P \parallel Q) = \Sigma P_i \log(P_i / Q_i)$.

The calibration set consists of 500–1000 training images sampled uniformly over scenes and crowd densities, pre-processed exactly as at inference (letter-boxing, normalisation) but without augmentation. Calibration on augmented images would produce mosaic-specific statistics that do not match deployment.

4.5.3 Quantisation-aware training

QAT inserts fake-quantisation operators that quantise and immediately de-quantise weights and activations in the forward pass, so the network is trained under quantisation noise. The non-differentiable rounding is handled by the straight-through estimator, which treats the derivative of round(·) as 1 inside the clipping range and 0 outside:

$$\partial \hat{r} / \partial r \approx 1 \text{ if } q_{min} \leq round(r/S) + Z \leq q_{max}, \text{ and 0 otherwise} \tag{4.6}$$

QAT starts from the converged FP32 model with BN already folded or with BN statistics frozen after the first epoch, uses a learning rate about one hundredth of the initial training rate, and runs for 10–20 epochs. Range observers are updated with a moving average during the first epochs and then frozen.

4.5.4 Sensitivity of the single-class head to quantisation

The objectness output passes through a sigmoid and is compared with a score threshold, so errors near the threshold change the set of surviving boxes. The regression outputs are multiplied by the stride after decoding; a quantisation error of one step S in a P5 distance therefore becomes an error of 32 S pixels in the box. For this reason the layer-wise sensitivity is analysed by quantising one layer at a time and measuring the AP drop; layers with the largest drop are candidates for FP16 execution.

If PTQ causes an unacceptable accuracy drop, quantisation-aware training (QAT) is applied for a small number of fine-tuning epochs. Because the head predicts only five channels per location, the detection head is sensitive to quantisation; keeping the final 1 x 1 layers in FP16 is evaluated as a mixed-precision option.

4.6 Deployment Pipeline

The deployment pipeline consists of: video frame capture, letter-box resize and normalisation on GPU, network inference, decoding of distance predictions, single-class NMS (IoU threshold 0.6, score threshold 0.3), and drawing or forwarding results. The end-to-end latency, rather than only network inference time, is reported, since pre- and post-processing can be a significant fraction of the total on embedded devices.

4.6.1 Deployment details

Export. After training, the EMA weights are loaded, BN layers are folded, ECA modules are kept as global pooling, 1-D convolution and multiplication, and the fusion weights of the neck are normalised and folded into constants. The model is exported to ONNX (opset 13 or later) with a fixed input of 1 x 3 x 416 x 416. The decoding of distances into boxes is included in the graph so that the network outputs boxes and scores directly; NMS is either included through the inference engine's NMS plugin or executed separately.

Inference engines. On NVIDIA Jetson devices the ONNX graph is compiled with TensorRT into FP16 and INT8 engines; layer fusion merges convolution, bias and activation, and the builder selects the fastest kernel implementation for each layer on the specific device. On ARM CPUs (e.g. Raspberry Pi 5) the model is run with ONNX Runtime or NCNN, using four threads and the INT8 kernels of the engine. Engine construction is performed on the target device because kernel selection is hardware-specific.

Pre-processing. Letter-box resizing, BGR-to-RGB conversion, normalisation to [0, 1] and HWC-to-CHW transposition are fused into a single CUDA kernel on Jetson, or implemented with vectorised NEON operations on ARM, to avoid multiple passes over the image in memory.

Post-processing. Scores below the threshold are discarded before NMS, which typically reduces the 3 549 candidates to a few dozen. Single-class NMS is executed on the GPU through the engine plugin or on the CPU with a sorted greedy implementation. Boxes are mapped back to the original image coordinates by undoing the letter-box padding and scaling.

Pipelining. For video, capture, pre-processing, inference and post-processing are run in separate threads connected by bounded queues, so that the throughput is limited by the slowest stage rather than by the sum of the stages. Both the single-frame latency and the pipelined throughput are reported, as they answer different deployment questions.

Power measurement. On Jetson devices the power draw is read from the on-board power monitors during sustained inference, and energy per frame is computed as average power divided by throughput. The device power mode is fixed and recorded for every measurement.

4.7 Chapter Summary

This chapter described how LSD-Net is trained and compressed: the construction of a single-class pedestrian dataset, data augmentation, training hyper-parameters, feature and response distillation from a larger teacher, and INT8 quantisation for deployment.

Chapter 5
Experiments and Analysis

Note to the author: the tables in this chapter define the experimental protocol and the required comparisons. Cells marked [ ] must be filled with values from your own training runs on your GPU. Do not fill them with estimated or published numbers presented as your own results. The analysis paragraphs are written to be revised once your numbers are available.

5.1 Experimental Environment

Item	Specification
Training GPU	[your GPU model, memory]
CPU / RAM	[ ]
Embedded device	[e.g. NVIDIA Jetson Orin Nano / Raspberry Pi 5]
OS	[ ]
Software	Python 3.x, PyTorch 2.x, CUDA [ ], TensorRT [ ], ONNX Runtime [ ]

Table 5.1 Experimental environment.

5.2 Evaluation Protocol

Accuracy is measured on the CrowdHuman validation set (full-body boxes) and the COCO val2017 person subset. AP@0.5, AP@0.5:0.95 and MR^{-2} are reported. Speed is measured as the median latency over 1,000 frames after 100 warm-up frames, with batch size 1, on both the training GPU and the embedded device. All models use the same input resolution within each comparison, and all baselines are retrained on the same single-class dataset with their official settings for a fair comparison.

5.3 Comparison with Lightweight Detectors

Model	Params (M)	FLOPs (G)	AP@0.5	AP@0.5 :0.95	MR^{-2}	FPS (GPU)	FPS (edge)
MobileNetV2-SSDLite	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
NanoDet-Plus-m	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
YOLOv5-n	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
YOLOv8-n	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
LSD-Net-S (ours)	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
LSD-Net-M (ours)	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

Model	Params (M)	FLOPs (G)	AP@0.5	AP@0.5 :0.95	MR^{-2}	FPS (GPU)	FPS (edge)
LSD-Net-L (ours)	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

Table 5.2 Comparison with lightweight detectors on the single-class pedestrian task (CrowdHuman val).

The analysis should discuss: (1) the accuracy–speed Pareto front, plotted as AP@0.5:0.95 against edge-device latency (Figure 5.1); (2) whether LSD-Net achieves comparable AP with fewer FLOPs; (3) the behaviour on crowded scenes, where MR^{-2} is most informative.

[Figure 5.1 placeholder — insert diagram or result image here]

Figure 5.1 Accuracy versus latency on the embedded device for all compared models.

5.4 Ablation Study

The ablation study starts from a baseline (MobileNetV2 backbone, PANet neck, multi-class-style head trained with a single class) and adds components one at a time.

Setting	ECA	Lite-BiFPN	Single quality head	Dynamic assign.	KD	AP@0.5 :0.95	FLOPs (G)
Baseline						[ ]	[ ]
+ ECA	✓					[ ]	[ ]
+ Lite-BiFPN	✓	✓				[ ]	[ ]
+ Quality head	✓	✓	✓			[ ]	[ ]
+ Dynamic assignment	✓	✓	✓	✓		[ ]	[ ]
+ Distillation (full)	✓	✓	✓	✓	✓	[ ]	[ ]

Table 5.3 Ablation of the components of LSD-Net-M.

Additional ablations to run:

- **Loss function**: IoU vs GIoU vs DIoU vs CIoU for box regression.
- **Neck width**: 48, 64, 96 channels.

- **Input resolution**: 320, 416, 512.
- **Distillation terms**: feature only, response only, both.

5.4.1 Loss function ablation

Box loss	Quality target	AP@0.5	AP@0.5:0.95	MR^{-2}
IoU	IoU (QFL)	[ ]	[ ]	[ ]
GIoU	IoU (QFL)	[ ]	[ ]	[ ]
DIoU	IoU (QFL)	[ ]	[ ]	[ ]
CIoU	IoU (QFL)	[ ]	[ ]	[ ]
CIoU	binary (focal loss)	[ ]	[ ]	[ ]
CIoU	binary + centre-ness branch	[ ]	[ ]	[ ]
CIoU, IoU-weighted positives	IoU (QFL)	[ ]	[ ]	[ ]

Table 5.4 Ablation of box regression loss and quality target (LSD-Net-M).

The loss ablation distinguishes two effects: the choice of the IoU-based regression loss and the semantics of the score. If the quality-target hypothesis of Chapter 3 holds, the gain of QFL over a binary target should be larger at AP@0.5:0.95 than at AP@0.5.

5.4.2 Neck and head ablation

Neck	Width C_n	Passes	Head	Params (M)	FLOPs (G)	AP@0.5 :0.95	Latency (ms, edge)
PAN (concat)	64	1	shared	[ ]	[ ]	[ ]	[ ]
Lite-BiFPN	48	1	shared	[ ]	[ ]	[ ]	[ ]
Lite-BiFPN	64	1	shared	[ ]	[ ]	[ ]	[ ]
Lite-BiFPN	96	1	shared	[ ]	[ ]	[ ]	[ ]
Lite-BiFPN	64	2	shared	[ ]	[ ]	[ ]	[ ]
Lite-BiFPN	64	1	decoupled	[ ]	[ ]	[ ]	[ ]
Lite-BiFPN	64	1	shared, 1 layer	[ ]	[ ]	[ ]	[ ]

Table 5.5 Ablation of neck type, width, number of passes and head structure.

5.4.3 Backbone ablation

Variant	ECA	Kernel (stages 3–4)	Activation	SPPF	FLOPs (G)	AP@0.5 :0.95	Latency (ms, edge)
LSD-Net-M	✓	5x5	h-swish	✓	[ ]	[ ]	[ ]

Variant	ECA	Kernel (stages 3–4)	Activation	SPPF	FLOPs (G)	AP@0.5:0.95	Latency (ms, edge)
no ECA		5x5	h-swish	✓	[ ]	[ ]	[ ]
SE instead of ECA	SE	5x5	h-swish	✓	[ ]	[ ]	[ ]
3x3 kernels	✓	3x3	h-swish	✓	[ ]	[ ]	[ ]
ReLU everywhere	✓	5x5	ReLU	✓	[ ]	[ ]	[ ]
no SPPF	✓	5x5	h-swish		[ ]	[ ]	[ ]
re-parameterised DW	✓	5x5 + 1x1 branch	h-swish	✓	[ ]	[ ]	[ ]

Table 5.6 Ablation of backbone design choices.

5.4.4 Label assignment ablation

Assignment	AP@0.5	AP@0.5:0.95	MR^{-2}	Avg. positives per object
FCOS (inside box + scale ranges)	[ ]	[ ]	[ ]	[ ]
Centre sampling only (radius 2.5)	[ ]	[ ]	[ ]	[ ]
ATSS	[ ]	[ ]	[ ]	[ ]
Task-aligned (TOOD)	[ ]	[ ]	[ ]	[ ]
Dynamic top-k (ours, without conflict resolution)	[ ]	[ ]	[ ]	[ ]
Dynamic top-k (ours)	[ ]	[ ]	[ ]	[ ]

Table 5.7 Comparison of label assignment strategies.

The average number of positives per object should be reported together with accuracy, because it explains differences between strategies: too few positives slow convergence, while too many introduce low-quality positives at object borders, which is especially harmful in crowded scenes.

5.4.5 Distillation ablation

Teacher	Feature KD	Response KD	Localisation KD	α_{bg}	AP@0.5:0.95	Training time (h)
none				–	[ ]	[ ]
LSD-style large	✓			0.25	[ ]	[ ]
LSD-style large		✓		–	[ ]	[ ]
LSD-style large	✓	✓		0.25	[ ]	[ ]
LSD-style large	✓	✓	✓	0.25	[ ]	[ ]
LSD-style large	✓	✓		0 (mask only)	[ ]	[ ]

Teacher	Feature KD	Response KD	Localisation KD	α_{bg}	AP@0.5:0.95	Training time (h)
LSD-style large	✓	✓		1.0 (no mask)	[ ]	[ ]
YOLOv8-m (retrained)	✓	✓		0.25	[ ]	[ ]

Table 5.8 Ablation of distillation components and teacher choice (student LSD-Net-M; teacher AP@0.5:0.95 = [ ]).

5.4.6 Hyper-parameter sensitivity

The following tables measure the sensitivity of LSD-Net-M to its main hyper-parameters. In each table only one hyper-parameter is varied and all others are kept at their default values. A method that depends critically on a finely tuned value is less useful for new single-class tasks, so flat sensitivity curves are desirable.

Centre radius (strides)	1.5	2.0	2.5	3.0	3.5
AP@0.5:0.95	[ ]	[ ]	[ ]	[ ]	[ ]
MR^{-2}	[ ]	[ ]	[ ]	[ ]	[ ]

Table 5.9 Sensitivity to the centre-sampling radius.

λ_{box}	1.0	1.5	2.0	3.0	5.0
AP@0.5	[ ]	[ ]	[ ]	[ ]	[ ]
AP@0.5:0.95	[ ]	[ ]	[ ]	[ ]	[ ]

Table 5.10 Sensitivity to the box loss weight ($\lambda_{obj} = 1.0$).

QFL β	0	1	2	3
AP@0.5:0.95	[ ]	[ ]	[ ]	[ ]

Table 5.11 Sensitivity to the focusing parameter β of the quality focal loss.

Distillation T / λ_{kd}	$\lambda_{kd} = 0.5$	$\lambda_{kd} = 1.0$	$\lambda_{kd} = 2.0$
T = 1	[ ]	[ ]	[ ]
T = 2	[ ]	[ ]	[ ]
T = 4	[ ]	[ ]	[ ]

Table 5.12 Sensitivity of AP@0.5:0.95 to distillation temperature and weight.

Optimiser / schedule	Initial LR	AP@0.5:0.95
SGD, cosine	0.01	[ ]
SGD, cosine	0.02	[ ]
SGD, step (x0.1 at 70 % and 90 %)	0.01	[ ]
AdamW, cosine	0.001	[ ]

Optimiser / schedule	Initial LR	AP@0.5:0.95
AdamW, cosine	0.002	[ ]

Table 5.13 Sensitivity to the optimiser and learning-rate schedule.

Epochs without mosaic at the end	0	5	15	30
AP@0.5:0.95	[ ]	[ ]	[ ]	[ ]

Table 5.14 Effect of disabling mosaic augmentation in the final epochs.

5.4.7 Input resolution and post-processing

Input size	FLOPs (G)	AP@0.5	AP@0.5:0.95	AP (small)	Latency (ms, edge)
320 x 320	[ ]	[ ]	[ ]	[ ]	[ ]
416 x 416	[ ]	[ ]	[ ]	[ ]	[ ]
512 x 512	[ ]	[ ]	[ ]	[ ]	[ ]
640 x 640	[ ]	[ ]	[ ]	[ ]	[ ]

Table 5.15 Effect of input resolution on LSD-Net-M.

NMS variant	IoU threshold	AP@0.5	AP@0.5:0.95	MR^{-2}	NMS time (ms, edge)
Greedy NMS	0.5	[ ]	[ ]	[ ]	[ ]
Greedy NMS	0.6	[ ]	[ ]	[ ]	[ ]
Greedy NMS	0.7	[ ]	[ ]	[ ]	[ ]
Soft-NMS (Gaussian, σ = 0.5)	–	[ ]	[ ]	[ ]	[ ]
DIoU-NMS	0.6	[ ]	[ ]	[ ]	[ ]

Table 5.16 Comparison of NMS variants on CrowdHuman val.

The NMS comparison is particularly informative on CrowdHuman, where the trade-off between suppressing duplicates and preserving overlapping true positives is most pronounced. The results should be broken down by crowd density (number of persons per image) if the differences are small on the full set.

5.5 Quantisation Results

Precision	Model size (MB)	AP@0.5:0.95	Latency (ms, edge)	FPS (edge)
FP32	[ ]	[ ]	[ ]	[ ]
FP16	[ ]	[ ]	[ ]	[ ]
INT8 (PTQ)	[ ]	[ ]	[ ]	[ ]
INT8 (QAT)	[ ]	[ ]	[ ]	[ ]

Table 5.17 Effect of quantisation on LSD-Net-M.

5.5.1 Layer-wise quantisation sensitivity and calibration

Calibration method	Calibration images	AP@0.5:0.95 (INT8 PTQ)
Min–max	500	[ ]
Percentile 99.99	500	[ ]
Entropy (KL)	100	[ ]
Entropy (KL)	500	[ ]
Entropy (KL)	1000	[ ]

Table 5.18 Effect of calibration method and calibration set size.

Configuration	AP@0.5:0.95	Latency (ms, edge)
INT8 per-tensor weights	[ ]	[ ]
INT8 per-channel weights	[ ]	[ ]
INT8, head output layers in FP16	[ ]	[ ]
INT8, stem and head output layers in FP16	[ ]	[ ]

Table 5.19 Effect of quantisation granularity and mixed precision.

A layer-wise sensitivity plot should show the AP drop when each layer alone is quantised to INT8 while the rest of the network stays in FP16. It is expected, but must be verified, that depthwise layers under per-tensor quantisation and the final regression layer are the most sensitive.

[Figure 5.2 placeholder — insert diagram or result image here]

Figure 5.2 Layer-wise INT8 sensitivity of LSD-Net-M (AP drop per layer).

5.5.2 Cross-platform deployment and energy

Device	Engine	Precision	Pre-proc (ms)	Inference (ms)	NMS (ms)	End-to-end FPS	Energy per frame (mJ)
Desktop GPU	TensorRT	FP16	[ ]	[ ]	[ ]	[ ]	–

Device	Engine	Precision	Pre-proc (ms)	Inference (ms)	NMS (ms)	End-to-end FPS	Energy per frame (mJ)
Jetson	TensorRT	FP16	[ ]	[ ]	[ ]	[ ]	[ ]
Jetson	TensorRT	INT8	[ ]	[ ]	[ ]	[ ]	[ ]
ARM CPU	ONNX Runtime	FP32	[ ]	[ ]	[ ]	[ ]	[ ]
ARM CPU	NCNN	INT8	[ ]	[ ]	[ ]	[ ]	[ ]

Table 5.20 Breakdown of end-to-end latency and energy for LSD-Net-M on different platforms.

5.5.3 Generalisation to other single-class tasks

To test whether the design generalises beyond pedestrians, LSD-Net-M and the strongest baseline are retrained without architectural changes on two additional single-class tasks with different shape statistics: faces (WIDER FACE) and vehicles (cars, vans and trucks of KITTI merged into one class).

Dataset	Model	AP@0.5	AP@0.5:0.95	Notes
WIDER FACE (val, easy / medium / hard AP)	Baseline	[ ]	–	[ ]
WIDER FACE (val, easy / medium / hard AP)	LSD-Net-M	[ ]	–	[ ]
KITTI vehicle	Baseline	[ ]	[ ]	[ ]
KITTI vehicle	LSD-Net-M	[ ]	[ ]	[ ]

Table 5.21 Generalisation of LSD-Net to other single-class domains.

5.5.4 Statistical reliability

Small detectors trained with strong augmentation can show run-to-run variation comparable to the gains of individual components. Each key configuration in the ablation (baseline, full model and each single-component removal) should therefore be trained with at least three random seeds, and the mean and standard deviation reported. Differences smaller than about twice the standard deviation should not be interpreted as improvements.

Configuration	Seed 1	Seed 2	Seed 3	Mean ± std
Baseline	[ ]	[ ]	[ ]	[ ]
LSD-Net-M (full)	[ ]	[ ]	[ ]	[ ]

Table 5.22 Run-to-run variation of AP@0.5:0.95.

5.6 Qualitative Analysis

Representative detection results should be shown for: sparse scenes, crowded scenes with heavy occlusion, small distant pedestrians, and low-light scenes (Figure 5.2). Typical failure cases — such as merged boxes for adjacent pedestrians, missed heavily occluded people and false positives on pedestrian-like structures (mannequins, posters) — should be shown and discussed (Figure 5.3).

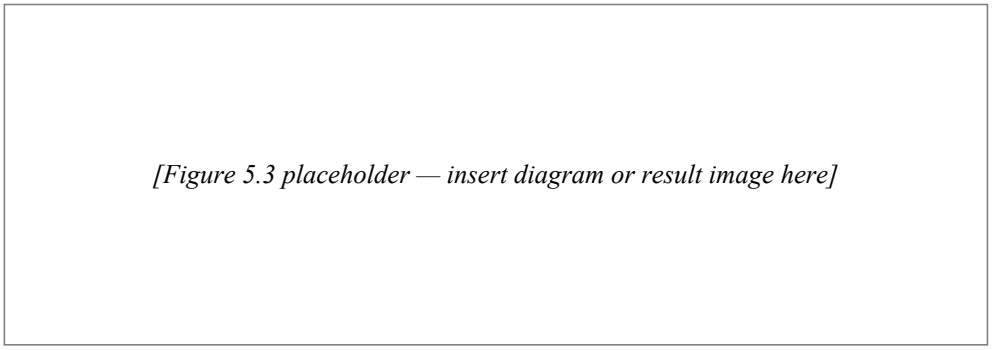


Figure 5.3 Qualitative detection results of LSD-Net in different scenes.

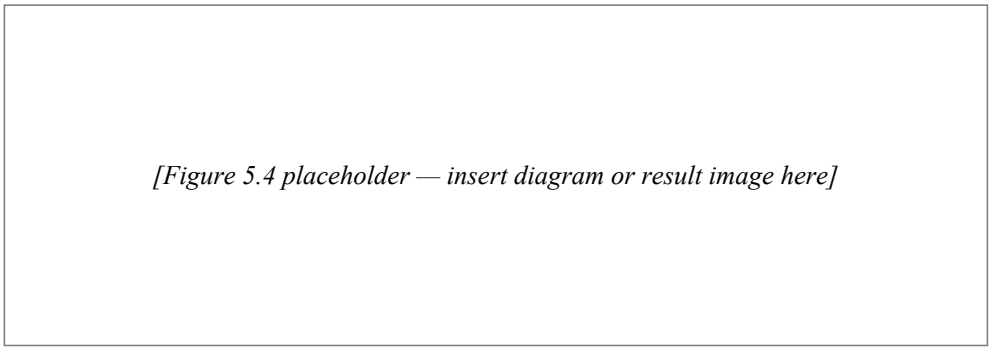


Figure 5.4 Typical failure cases and their causes.

5.7 Discussion

The discussion should relate the results to the design principles of Chapter 3. Points to address include: whether replacing the classification branch with a quality score improves high-IoU AP; whether weighted-addition fusion gives a better accuracy–cost trade-off than concatenation; how much of the gap to the teacher is closed by distillation; and how the measured edge latency compares with the FLOP count, since FLOPs do not always predict real latency because of memory access cost and operator support on the target hardware.

5.8 Chapter Summary

This chapter defined the experimental environment, the evaluation protocol, the comparison with lightweight baselines, the ablation study, the quantisation experiments and the qualitative analysis used to validate LSD-Net.

Chapter 6
Conclusion and Future Work

6.1 Conclusions

This thesis studied the design and implementation of a lightweight deep neural network for single-class object detection. The main work is summarised as follows:

- The redundancy of general multi-class detectors in single-class applications was analysed, and design principles for a single-class lightweight detector were derived.
- A lightweight backbone based on inverted residual blocks with efficient channel attention and a fast spatial pyramid pooling module was designed.
- A Lite-BiFPN neck using weighted addition was proposed for low-cost bidirectional multi-scale fusion.
- An anchor-free, shared, single-class head predicting an IoU-aware quality score was designed, trained with quality focal loss, CIoU loss and dynamic label assignment.
- A training pipeline combining mosaic augmentation, feature and response knowledge distillation and INT8 quantisation was developed, enabling deployment on embedded hardware.
- A complete experimental protocol including comparisons, ablations and quantisation analysis was designed to validate the contribution of each component.

6.2 Limitations

- Experiments focus on pedestrians; other single-class tasks with different shape statistics (e.g., small defects) may require different neck and resolution settings.
- The network is designed manually; neural architecture search might find better trade-offs.
- Heavy occlusion in very crowded scenes remains difficult for any dense detector that relies on NMS.

6.3 Future Work

- Apply hardware-aware neural architecture search to optimise the block configuration directly for latency on the target device.
- Investigate NMS-free training (one-to-one assignment) to remove post-processing cost and improve crowded-scene performance.

- Extend the method to other single-class tasks, such as helmets, vehicles or industrial defects, and study transfer learning from the pedestrian model.
- Integrate the detector with a multi-object tracker for real-time video analytics.

References

[1] Girshick R, Donahue J, Darrell T, Malik J. Rich feature hierarchies for accurate object detection and semantic segmentation. CVPR, 2014.

[2] Girshick R. Fast R-CNN. ICCV, 2015.

[3] Ren S, He K, Girshick R, Sun J. Faster R-CNN: Towards real-time object detection with region proposal networks. NeurIPS, 2015.

[4] Redmon J, Divvala S, Girshick R, Farhadi A. You only look once: Unified, real-time object detection. CVPR, 2016.

[5] Liu W, Anguelov D, Erhan D, et al. SSD: Single shot multibox detector. ECCV, 2016.

[6] Lin T Y, Goyal P, Girshick R, He K, Dollár P. Focal loss for dense object detection. ICCV, 2017.

[7] Lin T Y, Dollár P, Girshick R, et al. Feature pyramid networks for object detection. CVPR, 2017.

[8] Tian Z, Shen C, Chen H, He T. FCOS: Fully convolutional one-stage object detection. ICCV, 2019.

[9] Carion N, Massa F, Synnaeve G, et al. End-to-end object detection with transformers. ECCV, 2020.

[10] Howard A G, Zhu M, Chen B, et al. MobileNets: Efficient convolutional neural networks for mobile vision applications. arXiv:1704.04861, 2017.

[11] Sandler M, Howard A, Zhu M, Zhmoginov A, Chen L C. MobileNetV2: Inverted residuals and linear bottlenecks. CVPR, 2018.

[12] Howard A, Sandler M, Chu G, et al. Searching for MobileNetV3. ICCV, 2019.

[13] Ma N, Zhang X, Zheng H T, Sun J. ShuffleNet V2: Practical guidelines for efficient CNN architecture design. ECCV, 2018.

[14] Han K, Wang Y, Tian Q, et al. GhostNet: More features from cheap operations. CVPR, 2020.

[15] Tan M, Le Q. EfficientNet: Rethinking model scaling for convolutional neural networks. ICML, 2019.

[16] Tan M, Pang R, Le Q V. EfficientDet: Scalable and efficient object detection. CVPR, 2020.

[17] Hu J, Shen L, Sun G. Squeeze-and-excitation networks. CVPR, 2018.

[18] Wang Q, Wu B, Zhu P, et al. ECA-Net: Efficient channel attention for deep convolutional neural networks. CVPR, 2020.

[19] Zheng Z, Wang P, Liu W, et al. Distance-IoU loss: Faster and better learning for bounding box regression. AAAI, 2020.

[20] Li X, Wang W, Wu L, et al. Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection. NeurIPS, 2020.

[21] Ge Z, Liu S, Wang F, Li Z, Sun J. YOLOX: Exceeding YOLO series in 2021. arXiv:2107.08430, 2021.

[22] Hinton G, Vinyals O, Dean J. Distilling the knowledge in a neural network. arXiv:1503.02531, 2015.

[23] Jacob B, Kligys S, Chen B, et al. Quantization and training of neural networks for efficient integer-arithmetic-only inference. CVPR, 2018.

[24] Shao S, Zhao Z, Li B, et al. CrowdHuman: A benchmark for detecting human in a crowd. arXiv:1805.00123, 2018.

[25] Lin T Y, Maire M, Belongie S, et al. Microsoft COCO: Common objects in context. ECCV, 2014.

[26] Bazarevsky V, Kartynnik Y, Vakunov A, et al. BlazeFace: Sub-millisecond neural face detection on mobile GPUs. arXiv:1907.05047, 2019.

[27] He K, Zhang X, Ren S, Sun J. Deep residual learning for image recognition. CVPR, 2016.

[28] Dollár P, Wojek C, Schiele B, Perona P. Pedestrian detection: An evaluation of the state of the art. IEEE TPAMI, 2012.

[29] Redmon J, Farhadi A. YOLO9000: Better, faster, stronger. CVPR, 2017.

[30] Redmon J, Farhadi A. YOLOv3: An incremental improvement. arXiv:1804.02767, 2018.

[31] Bochkovskiy A, Wang C Y, Liao H Y M. YOLOv4: Optimal speed and accuracy of object detection. arXiv:2004.10934, 2020.

[32] Li C, Li L, Jiang H, et al. YOLOv6: A single-stage object detection framework for industrial applications. arXiv:2209.02976, 2022.

[33] Wang C Y, Bochkovskiy A, Liao H Y M. YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. CVPR, 2023.

[34] Wang C Y, Yeh I H, Liao H Y M. YOLOv9: Learning what you want to learn using programmable gradient information. ECCV, 2024.

[35] Wang A, Chen H, Liu L, et al. YOLOv10: Real-time end-to-end object detection. NeurIPS, 2024.

[36] He K, Gkioxari G, Dollár P, Girshick R. Mask R-CNN. ICCV, 2017.

[37] Cai Z, Vasconcelos N. Cascade R-CNN: Delving into high quality object detection. CVPR, 2018.

[38] Zhou X, Wang D, Krähenbühl P. Objects as points. arXiv:1904.07850, 2019.

[39] Law H, Deng J. CornerNet: Detecting objects as paired keypoints. ECCV, 2018.

[40] Zhu X, Su W, Lu L, et al. Deformable DETR: Deformable transformers for end-to-end object detection. ICLR, 2021.

[41] Zhao Y, Lv W, Xu S, et al. DETRs beat YOLOs on real-time object detection. CVPR, 2024.

[42] Zhang S, Chi C, Yao Y, Lei Z, Li S Z. Bridging the gap between anchor-based and anchor-free detection via adaptive training sample selection. CVPR, 2020.

[43] Feng C, Zhong Y, Gao Y, Scott M R, Huang W. TOOD: Task-aligned one-stage object detection. ICCV, 2021.

[44] Zhang H, Wang Y, Dayoub F, Sünderhauf N. VarifocalNet: An IoU-aware dense object detector. CVPR, 2021.

[45] Rezatofighi H, Tsoi N, Gwak J, et al. Generalized intersection over union: A metric and a loss for bounding box regression. CVPR, 2019.

[46] Bodla N, Singh B, Chellappa R, Davis L S. Soft-NMS: Improving object detection with one line of code. ICCV, 2017.

[47] Liu S, Qi L, Qin H, Shi J, Jia J. Path aggregation network for instance segmentation. CVPR, 2018.

[48] Zhang X, Zhou X, Lin M, Sun J. ShuffleNet: An extremely efficient convolutional neural network for mobile devices. CVPR, 2018.

[49] Mehta S, Rastegari M. MobileViT: Light-weight, general-purpose, and mobile-friendly vision transformer. ICLR, 2022.

[50] Ding X, Zhang X, Ma N, et al. RepVGG: Making VGG-style ConvNets great again. CVPR, 2021.

[51] Yu G, Chang Q, Lv W, et al. PP-PicoDet: A better real-time object detector on mobile devices. arXiv:2111.00902, 2021.

[52] Han S, Mao H, Dally W J. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. ICLR, 2016.

[53] Liu Z, Li J, Shen Z, et al. Learning efficient convolutional networks through network slimming. ICCV, 2017.

[54] Li H, Kadav A, Durdanovic I, Samet H, Graf H P. Pruning filters for efficient ConvNets. ICLR, 2017.

[55] Chen G, Choi W, Yu X, Han T, Chandraker M. Learning efficient object detection models with knowledge distillation. NeurIPS, 2017.

[56] Wang T, Yuan L, Zhang X, Feng J. Distilling object detectors with fine-grained feature imitation. CVPR, 2019.

[57] Zheng Z, Ye R, Wang P, et al. Localization distillation for dense object detection. CVPR, 2022.

[58] Romero A, Ballas N, Kahou S E, et al. FitNets: Hints for thin deep nets. ICLR, 2015.

[59] Ioffe S, Szegedy C. Batch normalization: Accelerating deep network training by reducing internal covariate shift. ICML, 2015.

[60] Kingma D P, Ba J. Adam: A method for stochastic optimization. ICLR, 2015.

[61] Loshchilov I, Hutter F. Decoupled weight decay regularization. ICLR, 2019.

[62] Rumelhart D E, Hinton G E, Williams R J. Learning representations by back-propagating errors. Nature, 1986.

[63] Zhang S, Benenson R, Schiele B. CityPersons: A diverse dataset for pedestrian detection. CVPR, 2017.

[64] Yang S, Luo P, Loy C C, Tang X. WIDER FACE: A face detection benchmark. CVPR, 2016.

[65] Deng J, Guo J, Ververas E, Kotsia I, Zafeiriou S. RetinaFace: Single-shot multi-level face localisation in the wild. CVPR, 2020.

[66] Geiger A, Lenz P, Urtasun R. Are we ready for autonomous driving? The KITTI vision benchmark suite. CVPR, 2012.

[67] Dalal N, Triggs B. Histograms of oriented gradients for human detection. CVPR, 2005.

[68] Felzenszwalb P F, Girshick R B, McAllester D, Ramanan D. Object detection with discriminatively trained part-based models. IEEE TPAMI, 2010.

[69] Krishnamoorthi R. Quantizing deep convolutional networks for efficient inference: A whitepaper. arXiv:1806.08342, 2018.

[70] Everingham M, Van Gool L, Williams C K I, Winn J, Zisserman A. The PASCAL visual object classes (VOC) challenge. IJCV, 2010.

[71] Zhang H, Cisse M, Dauphin Y N, Lopez-Paz D. mixup: Beyond empirical risk minimization. ICLR, 2018.

[72] Ramachandran P, Zoph B, Le Q V. Searching for activation functions. arXiv:1710.05941, 2017.

[73] Wang C Y, Liao H Y M, Wu Y H, et al. CSPNet: A new backbone that can enhance learning capability of CNN. CVPR Workshops, 2020.

[74] Luo W, Li Y, Urtasun R, Zemel R. Understanding the effective receptive field in deep convolutional neural networks. NeurIPS, 2016.

Acknowledgements

[Write your acknowledgements here — supervisor, laboratory colleagues, family and funding sources.]